

Self-Attention and Cross-Attention via Mamba-3, and Their Application to Images

Masahiro Ogawa

May 19, 2026

Abstract

This note derives, step by step, how the Mamba-3 state space model (SSM) can be used as a drop-in replacement for both self-attention and cross-attention, and how the resulting attention blocks are assembled into an image model. We follow the notation of the original Mamba-3 paper so every equation can be checked against the source. The guiding principle is that every algebraic move is shown with its justification. A reader who knows matrix multiplication and elementary calculus should be able to follow the whole note without guessing.

Contents

1	Preliminaries and Notation	2
1.1	Sizes	2
1.2	Tokens and projections	3
1.3	RMSNorm and BCNorm	4
1.4	Recurrence, parallel form, and decay factors	4
1.5	What this note actually does	4
2	Why SSD $\equiv$ linear attention (the duality, derived)	5
2.1	Unrolling the two-term recurrence	5
2.2	From state to output	5
2.3	Matrix form: $\mathbf{Y} = (\mathbf{L} \odot \mathbf{C}\mathbf{B}^\top)\mathbf{X}$	6
2.4	Reading the SSD identity as linear attention	6
2.5	Trapezoidal three-term form	6
3	Self-Attention via Mamba-3	7
3.1	Per-token projections	8
3.2	Self-attention output	8
3.3	Complex-valued SSM / RoPE trick: attention with learned RoPE	9
3.4	MIMO extension	9
3.5	Worked example: self-attention on one 504×504 image	9
4	Cross-Attention via Mamba-3	10
4.1	Projections for the two sides	11
4.2	Variant A: state-compressed cross-attention (constant memory)	11
4.3	Variant B: token-level cross-attention (exact SSD)	12
4.4	Which variant to use	13
4.5	Worked example: cross-attention between two 504×504 images	13

5	Applying Mamba-3 to Images	13
5.1	Patchification	14
5.2	Why a plain causal scan fails on images	14
5.3	Bidirectional and multi-direction scans	14
5.4	2-D positional encoding via the complex SSM	15
5.5	Image self-attention layer	15
5.6	Multi-view cross-attention for 3-D applications	15
6	Non-Causal SSD and the VSSD Image Model	16
6.1	The reinterpretation of A_t	16
6.2	Bidirectional NC-SSD collapses to a global hidden state	16
6.3	Closed form: $\mathbf{Y} = \mathbf{C}((\mathbf{B} \odot \mathbf{m})^\top \mathbf{X})$	17
6.4	Relation to Mamba-3	17
6.5	Tensor-contraction algorithm	18
6.6	The VSSD image block	18
6.7	Worked example: VSSD on one 504×504 image	18
6.8	When to use NC-SSD versus bidirectional SSD	19
7	Application: From the Token Tensor to a Segmentation Mask	20
7.1	Head A: linear projection + bilinear upsample	20
7.2	Head B: multi-scale DPT-style decoder	20
7.3	Head C: mask-classification with learnable queries	21
7.4	Why the attention variant is interchangeable	21
8	Application: Multi-Frame 3D Point Tracking	22
8.1	Architecture overview	22
8.2	Within-frame block: VSSD self-attention	23
8.3	Across-frame block: causal cross-SSD	23
8.4	Per-track read-out heads	23
8.5	Spawn and disappear	24
8.6	Loss	24
8.7	Why this operator pairing	26
9	End-to-End Summary	26

1 Preliminaries and Notation

We summarize the symbols used throughout. The names match [1], Section 2 and Section 3.

1.1 Sizes

- T : sequence length (number of tokens). For images we will set $T = L = HW/P^2$ where H, W are the image height and width and P is the patch size.
- D : model (channel) dimension.
- N : SSM state dimension (per SSM head).
- R : MIMO rank (Mamba-3, §3.3). For SISO, $R = 1$.

1.2 Tokens and projections

- $\mathbf{X} \in \mathbb{R}^{T \times D}$ is the input sequence; the t -th row $\mathbf{x}_t \in \mathbb{R}^D$ is one token.
- $\mathbf{B}, \mathbf{C} \in \mathbb{R}^{T \times N}$ are data-dependent projections; the t -th rows are $\mathbf{B}_t, \mathbf{C}_t \in \mathbb{R}^N$.
- $\Delta_t \in \mathbb{R}_{>0}$ is a data-dependent step size.
- $A_t \in \mathbb{R}_{<0}$ is a data-dependent (in Mamba-3) or data-independent (in Mamba-2) scalar that governs decay.
- $\mathbf{h}_t \in \mathbb{R}^N$ is the hidden state at time t (SISO; see §3.4 for MIMO).
- $\mathbf{Y} \in \mathbb{R}^{T \times D}$ is the output sequence.

In terms of an input token $\mathbf{x}_t$, the learned projections are

$$\mathbf{B}_t = \mathbf{W}_B \mathbf{x}_t \quad \mathbf{W}_B \in \mathbb{R}^{N \times D}, \quad (1)$$

$$\mathbf{C}_t = \mathbf{W}_C \mathbf{x}_t \quad \mathbf{W}_C \in \mathbb{R}^{N \times D}, \quad (2)$$

$$\Delta_t = \text{Softplus}(\mathbf{w}_\Delta^\top \mathbf{x}_t) \quad \mathbf{w}_\Delta \in \mathbb{R}^D, \quad (3)$$

$$A_t = -\text{Softplus}(\mathbf{w}_A^\top \mathbf{x}_t) \quad \mathbf{w}_A \in \mathbb{R}^D \text{ (optional data-dep.)}. \quad (4)$$

Here Softplus is the smooth, strictly positive activation

$$\text{Softplus}(z) := \log(1 + e^z), \quad \text{Softplus} : \mathbb{R} \rightarrow \mathbb{R}_{>0}, \quad (5)$$

so (3) guarantees $\Delta_t > 0$ and (4) guarantees $A_t < 0$, which is what the discretization in (12) requires for $\alpha_t = e^{\Delta_t A_t} \in (0, 1)$.

The value projection that we will feed into the SSM is

$$\mathbf{v}_t = \text{SiLU}(\mathbf{W}_V \mathbf{x}_t), \quad \mathbf{W}_V \in \mathbb{R}^{D \times D}, \quad (6)$$

where SiLU (a.k.a. Swish) is the smooth, self-gated activation

$$\text{SiLU}(z) := z \sigma(z) = \frac{z}{1 + e^{-z}}, \quad \sigma(z) := \frac{1}{1 + e^{-z}}, \quad (7)$$

applied element-wise to the vector $\mathbf{W}_V \mathbf{x}_t \in \mathbb{R}^D$.

We also use softmax when comparing SSD to standard attention. For a row vector $\mathbf{z} = (z_1, \dots, z_T) \in \mathbb{R}^T$,

$$\text{softmax}(\mathbf{z})_j := \frac{e^{z_j}}{\sum_{k=1}^T e^{z_k}}, \quad j = 1, \dots, T, \quad (8)$$

so the entries of $\text{softmax}(\mathbf{z})$ are non-negative and sum to 1. Applied to a matrix $\mathbf{S} \in \mathbb{R}^{T \times T}$ as $\text{softmax}(\mathbf{S})$, the operator is taken *row-wise* — each row of $\mathbf{S}$ is independently mapped through (8), so each row of $\text{softmax}(\mathbf{S})$ sums to 1. In standard scaled-dot-product attention this is what enforces “each query’s attention weights form a probability distribution over the keys”:

$$\mathbf{Y} = \text{softmax}(\mathbf{Q}\mathbf{K}^\top / \sqrt{d}) \mathbf{V}.$$

The Mamba-3 SSD parallel form (14) replaces this row-wise softmax with element-wise multiplication by a learned mask $\mathbf{L}$, dropping both the exponential and the row normalisation.

1.3 RMSNorm and BCNorm

Mamba-3 normalizes the key/query projections $\mathbf{B}_t, \mathbf{C}_t$ before using them in the SSD kernel. The underlying primitive is *RMSNorm* [4], which rescales a vector by its root-mean-square (no mean subtraction, unlike LayerNorm):

$$\text{RMSNorm}(\mathbf{u}) := \frac{\mathbf{u}}{\sqrt{\frac{1}{N} \sum_{i=1}^N u_i^2 + \varepsilon}} \odot \mathbf{g}, \quad \mathbf{u} \in \mathbb{R}^N, \quad (9)$$

where $\mathbf{g} \in \mathbb{R}^N$ is a learnable per-channel gain, $\varepsilon > 0$ is a small constant, and $\odot$ is the Hadamard product. The output has unit RMS up to the learned gain, which stabilizes the inner products $\mathbf{C}_i^\top \mathbf{B}_j$ that appear in (14).

BCNorm ([1] §3.4) is simply RMSNorm applied independently to $\mathbf{B}_t$ and $\mathbf{C}_t$, followed by a learnable channel-wise bias:

$$\mathbf{B}_t \leftarrow \text{RMSNorm}(\mathbf{B}_t) + \mathbf{b}_B, \quad \mathbf{C}_t \leftarrow \text{RMSNorm}(\mathbf{C}_t) + \mathbf{b}_C, \quad \mathbf{b}_B, \mathbf{b}_C \in \mathbb{R}^N. \quad (10)$$

We will write “apply BCNorm” as shorthand for the substitution (10); each occurrence of $\mathbf{B}_t, \mathbf{C}_t$ in later equations is understood to be the post-BCNorm version when the algorithm calls for it.

1.4 Recurrence, parallel form, and decay factors

Mamba-3 uses the *exponential-trapezoidal* discrete recurrence of [1] eq. (5)–(6):

$$\mathbf{h}_t = \alpha_t \mathbf{h}_{t-1} + \beta_t \mathbf{B}_{t-1} \mathbf{x}_{t-1} + \gamma_t \mathbf{B}_t \mathbf{x}_t, \quad \mathbf{y}_t = \mathbf{C}_t^\top \mathbf{h}_t, \quad (11)$$

with the three data-dependent scalars

$$\alpha_t := e^{\Delta_t A_t}, \quad \beta_t := (1 - \lambda_t) \Delta_t e^{\Delta_t A_t}, \quad \gamma_t := \lambda_t \Delta_t, \quad (12)$$

where $\lambda_t \in [0, 1]$ is a learned, data-dependent convex weight. Setting $\lambda_t \equiv 1$ recovers Mamba-2’s two-term recurrence

$$\mathbf{h}_t = \alpha_t \mathbf{h}_{t-1} + \gamma_t \mathbf{B}_t \mathbf{x}_t, \quad \mathbf{y}_t = \mathbf{C}_t^\top \mathbf{h}_t, \quad (13)$$

which is [1] eq. (1). Mamba-3’s *parallel* form ([1] eq. (2)) reads

$$\mathbf{Y} = (\mathbf{L} \odot \mathbf{C} \mathbf{B}^\top) \mathbf{X}, \quad (14)$$

with $\odot$ the Hadamard product and $\mathbf{L} \in \mathbb{R}^{T \times T}$ a structured lower-triangular mask that we derive explicitly in Section 2. This single equation is the bridge between recurrences and attention: the next section is devoted to explaining exactly why.

1.5 What this note actually does

Using only the objects above, we will:

1. Derive (14) from (13) and (11), and read it as *linear attention with a learned mask* (Section 2).
2. Plug real projections into (14) to get a Mamba-3 self-attention block (Section 3).
3. Repeat for two different sequences and obtain Mamba-3 cross-attention, in two flavours: state-compressed (cheap, constant-memory) and token-level (exact SSD analogue of softmax cross-attention) (Section 4).
4. Patchify an image, make the scan bidirectional, and use the self/cross-attention blocks to build an image encoder; then combine several views via cross-view attention for the 3-D setting (Section 5).
5. Derive the *non-causal SSD* (NC-SSD) of [3] as an algebraic alternative to bidirectional scans, and show that it collapses the structured $T \times T$ mask $\mathbf{L}$ to a single learned T -vector $\mathbf{m}$ (Section 6).

2 Why SSD $\equiv$ linear attention (the duality, derived)

We derive (14) directly from the recurrence. Work entirely with the two-term form (13) first; the three-term form (11) is a corollary (§2.5).

2.1 Unrolling the two-term recurrence

Starting from $\mathbf{h}_0 = \mathbf{0}$ and applying (13) once:

$$\mathbf{h}_1 = \alpha_1 \mathbf{h}_0 + \gamma_1 \mathbf{B}_1 \mathbf{x}_1 = \gamma_1 \mathbf{B}_1 \mathbf{x}_1. \quad (15)$$

Apply it once more:

$$\begin{aligned} \mathbf{h}_2 &= \alpha_2 \mathbf{h}_1 + \gamma_2 \mathbf{B}_2 \mathbf{x}_2 \\ &= \alpha_2 \gamma_1 \mathbf{B}_1 \mathbf{x}_1 + \gamma_2 \mathbf{B}_2 \mathbf{x}_2. \end{aligned} \quad (16)$$

By induction on t we claim

$$\mathbf{h}_t = \sum_{j=1}^t \underbrace{\left(\prod_{k=j+1}^t \alpha_k \right)}_{=: a_{t \leftarrow j}} \gamma_j \mathbf{B}_j \mathbf{x}_j, \quad (17)$$

where by convention an empty product equals 1 (so $a_{t \leftarrow t} = 1$).

Proof of the induction step. Assume (17) at time $t-1$. Substitute into (13):

$$\mathbf{h}_t = \alpha_t \mathbf{h}_{t-1} + \gamma_t \mathbf{B}_t \mathbf{x}_t \quad (18)$$

$$= \alpha_t \sum_{j=1}^{t-1} \left(\prod_{k=j+1}^{t-1} \alpha_k \right) \gamma_j \mathbf{B}_j \mathbf{x}_j + \gamma_t \mathbf{B}_t \mathbf{x}_t \quad (19)$$

$$= \sum_{j=1}^{t-1} \left(\prod_{k=j+1}^t \alpha_k \right) \gamma_j \mathbf{B}_j \mathbf{x}_j + \gamma_t \mathbf{B}_t \mathbf{x}_t \quad (\text{absorb } \alpha_t \text{ into the product}) \quad (20)$$

$$= \sum_{j=1}^t \left(\prod_{k=j+1}^t \alpha_k \right) \gamma_j \mathbf{B}_j \mathbf{x}_j. \quad (21)$$

The last equality uses the empty-product convention for $j = t$. This proves (17).

2.2 From state to output

Plug (17) into the output equation $\mathbf{y}_t = \mathbf{C}_t^\top \mathbf{h}_t$:

$$\mathbf{y}_t = \mathbf{C}_t^\top \sum_{j=1}^t a_{t \leftarrow j} \gamma_j \mathbf{B}_j \mathbf{x}_j \quad (22)$$

$$= \sum_{j=1}^t a_{t \leftarrow j} \gamma_j \underbrace{\mathbf{C}_t^\top \mathbf{B}_j}_{\text{scalar}} \mathbf{x}_j \quad (\text{linearity of } \mathbf{C}_t^\top(\cdot)) \quad (23)$$

$$= \sum_{j=1}^t \underbrace{(a_{t \leftarrow j} \gamma_j)}_{=: L_{tj}} (\mathbf{C}_t^\top \mathbf{B}_j) \mathbf{x}_j. \quad (24)$$

The key observation is that $\mathbf{x}_j$ is a vector while $\mathbf{C}_t^\top \mathbf{B}_j$ is a scalar; we are free to reorder the factors.

2.3 Matrix form: $\mathbf{Y} = (\mathbf{L} \odot \mathbf{C}\mathbf{B}^\top)\mathbf{X}$

Define the lower-triangular matrix

$$L_{tj} = \begin{cases} \gamma_j \prod_{k=j+1}^t \alpha_k & j \leq t, \\ 0 & j > t. \end{cases} \quad (25)$$

Equivalently,

$$\mathbf{L} = \begin{bmatrix} 1 & & & \\ \alpha_2 & 1 & & \\ \alpha_3 \alpha_2 & \alpha_3 & 1 & \\ \vdots & & \ddots & \\ \prod_{k=2}^T \alpha_k & \cdots & \alpha_T & 1 \end{bmatrix} \cdot \text{diag}(\gamma_1, \dots, \gamma_T), \quad (26)$$

which is the Mamba-2 mask of [1] eq. (3). Using (24) row by row,

$$\mathbf{y}_t = \sum_{j=1}^T L_{tj} (\mathbf{C}_t^\top \mathbf{B}_j) \mathbf{x}_j = \sum_{j=1}^T [\mathbf{L} \odot \mathbf{C}\mathbf{B}^\top]_{tj} \mathbf{x}_j. \quad (27)$$

(Terms with $j > t$ vanish because $L_{tj} = 0$.) Stacking over t ,

$$\boxed{\mathbf{Y} = (\mathbf{L} \odot \mathbf{C}\mathbf{B}^\top)\mathbf{X}.} \quad (28)$$

This is exactly (14) ([1] eq. (2)).

2.4 Reading the SSD identity as linear attention

Define the standard attention quantities

$$\mathbf{Q}_t := \mathbf{C}_t, \quad \mathbf{K}_j := \mathbf{B}_j, \quad \mathbf{V}_j := \mathbf{x}_j. \quad (29)$$

Then (28) is

$$\mathbf{Y} = (\mathbf{L} \odot \mathbf{Q}\mathbf{K}^\top)\mathbf{V}, \quad (30)$$

which is *linear attention* (no softmax, no row normalization), with a data-dependent, structured mask $\mathbf{L}$. Compare to standard softmax self-attention $\mathbf{Y} = \text{softmax}(\mathbf{Q}\mathbf{K}^\top / \sqrt{d})\mathbf{V}$:

- Standard attention computes a dense $T \times T$ similarity matrix $\mathbf{Q}\mathbf{K}^\top$ and row-normalizes it by softmax.
- Mamba-3 SSD computes the same $\mathbf{Q}\mathbf{K}^\top$, then multiplies it elementwise by a learned mask $\mathbf{L}$ whose off-diagonal entries decay geometrically in $|t - j|$.
- The decay factor $\prod_{k=j+1}^t \alpha_k$ plays the role of a relative-position bias: tokens separated by large $\sum_k \Delta_k |A_k|$ contribute exponentially less.

2.5 Trapezoidal three-term form

The same derivation goes through for (11). We show only the new piece: the mask acquires an extra two-band contribution. Substitute (11) once,

$$\mathbf{h}_t = \alpha_t \mathbf{h}_{t-1} + \beta_t \mathbf{B}_{t-1} \mathbf{x}_{t-1} + \gamma_t \mathbf{B}_t \mathbf{x}_t. \quad (31)$$

Iterating (compare (17)) gives

$$\mathbf{h}_t = \sum_{j=1}^t \left(\prod_{k=j+1}^t \alpha_k \right) (\gamma_j \mathbf{B}_j \mathbf{x}_j + \beta_j \mathbf{B}_{j-1} \mathbf{x}_{j-1}), \quad (32)$$

where the β_j term contributes to the state via $\mathbf{B}_{j-1} \mathbf{x}_{j-1}$. Plugging into $\mathbf{y}_t = \mathbf{C}_t^\top \mathbf{h}_t$ and collecting coefficients of $\mathbf{B}_j \mathbf{x}_j$ (note the index shift for the β term), one obtains

$$L_{tj} = \left(\prod_{k=j+1}^t \alpha_k \right) \gamma_j + \left(\prod_{k=j+2}^t \alpha_k \right) \beta_{j+1} \quad \text{for } j \leq t, \quad (33)$$

with the usual empty-product convention. In matrix form this is [1] eq. (7): $\mathbf{L}$ is the product of a 1-semiseparable matrix (whose (t, j) entry is $\prod_{k=j+1}^t \alpha_k$) and a two-band matrix $\text{diag}(\gamma_0, \dots, \gamma_T) + \text{subdiag}(\beta_1, \dots, \beta_T)$. Equations (27)–(28) are unchanged.

Boundary conditions of the β band. The second term in (33) references β_{j+1} . For the left endpoint, the recurrence (11) at $t = 1$ uses $\beta_1 \mathbf{B}_0 \mathbf{x}_0$ with $\mathbf{B}_0, \mathbf{x}_0$ out of range; we therefore adopt the convention $\beta_1 := 0$, or equivalently drop this term when $t = 1$. Consistently, β_1 never appears in (33): the second term at $j = 0$ would need β_1 , but $j \geq 1$ by indexing. For the right endpoint, at $j = t$ the expression $\beta_{j+1} = \beta_{t+1}$ is undefined; we set $\beta_{T+1} := 0$ so the second term vanishes at $j = t$, leaving $L_{tt} = \gamma_t$ as expected. Any implementation that pads β with zeros on both ends and builds the second band only for the strictly lower triangle ($j < t$) honours both conventions automatically. An off-by-one in this shift is silently masked by the test $\lambda_t \equiv 1$ (which zeroes β identically) and must therefore be caught by an independent test with $\lambda_t \in (0, 1)$.

3 Self-Attention via Mamba-3

We now instantiate the duality (28) as a concrete self-attention block for a single sequence $\mathbf{X} \in \mathbb{R}^{T \times D}$.

Bridge from eq. (28) to the attention form $\mathbf{Y} = (\mathbf{L} \odot \mathbf{C}\mathbf{B}^\top) \mathbf{V}$. The SSD identity $\mathbf{Y} = (\mathbf{L} \odot \mathbf{C}\mathbf{B}^\top) \mathbf{X}$ was derived in §2 for the recurrence (11), in which the symbol $\mathbf{x}_t$ stands for *whatever sequence is fed into the recurrence*. The symbol $\mathbf{X}$ in (28) is therefore a *placeholder for the kernel’s input matrix*, not a name reserved for the raw patch-embedding sequence. In a Mamba-3 *attention* block the architectural choice is to feed not the raw token but the value-projected vector $\mathbf{v}_t = \text{SiLU}(\mathbf{W}_V \mathbf{x}_t)$ (6) into the recurrence:

$$\mathbf{h}_t = \alpha_t \mathbf{h}_{t-1} + \beta_t \mathbf{B}_{t-1} \mathbf{v}_{t-1} + \gamma_t \mathbf{B}_t \mathbf{v}_t, \quad \mathbf{y}_t = \mathbf{C}_t^\top \mathbf{h}_t. \quad (34)$$

The unrolling of §2 did not depend on what the input is called — only that the recurrence has the shape (11) — so (34) unrolls to exactly (28) with $\mathbf{X}$ replaced by $\mathbf{V}$:

$$\boxed{\mathbf{Y}_{\text{self}} = (\mathbf{L} \odot \mathbf{C}\mathbf{B}^\top) \mathbf{V}}, \quad \mathbf{V} := (\mathbf{v}_1, \dots, \mathbf{v}_T)^\top \in \mathbb{R}^{T \times D}. \quad (35)$$

This is the same identity (28), applied to the specific input the block hands the kernel.

Placing self-attention next to softmax attention:

$$\underbrace{\mathbf{Y} = (\mathbf{L} \odot \mathbf{C}\mathbf{B}^\top) \mathbf{V}}_{\text{Mamba SSD (2 or 3)}} \iff \underbrace{\mathbf{Y} = \text{softmax}(\mathbf{Q}\mathbf{K}^\top / \sqrt{d}) \mathbf{V}}_{\text{softmax attention}},$$

the substitution

$$\mathbf{Q} \leftarrow \mathbf{C}, \quad \mathbf{K} \leftarrow \mathbf{B}, \quad \mathbf{V} \leftarrow \text{SiLU}(\mathbf{W}_V \mathbf{x}),$$

makes the analogy literal: the *query* projection becomes $\mathbf{C}$ and the *key* projection becomes $\mathbf{B}$; the value $\mathbf{V}$ is the same one softmax attention uses. The only structural difference is that row normalisation $\text{softmax}(\cdot/\sqrt{d})$ has been replaced by an element-wise multiplication by the learned, structured mask $\mathbf{L}$. That is the whole operator: pick the right $\mathbf{Q}/\mathbf{K}/\mathbf{V}$, build $\mathbf{L}$ from the per-token scalars, multiply.

The trapezoidal three-term form does not change this shape. A reasonable worry: Mamba-3’s three-term recurrence (11) adds a β_t band that mixes adjacent tokens, so does the output formula still factor as $\mathbf{Y} = (\mathbf{L} \odot \mathbf{C}\mathbf{B}^\top)\mathbf{V}$? It does — only the *contents* of $\mathbf{L}$ change. §2.5 derived this already: collecting coefficients of $\mathbf{B}_j \mathbf{x}_j$ in the unrolled three-term state gives (33),

$$L_{tj} = \left(\prod_{k=j+1}^t \alpha_k \right) \gamma_j + \left(\prod_{k=j+2}^t \alpha_k \right) \beta_{j+1},$$

i.e. *two* contributions per (t, j) cell instead of one. The Mamba-2 two-term mask (25) is exactly the special case $\beta \equiv 0$ (equivalently $\lambda_t \equiv 1$). Plugging this richer L_{tj} into the same row formula (27) reproduces $\mathbf{Y} = (\mathbf{L} \odot \mathbf{C}\mathbf{B}^\top)\mathbf{V}$ unchanged; only the mask gets an extra band. Consequence for an implementer: Mamba-3 self-attention reuses every line of code that handles Mamba-2 SSD, with the single change of swapping the mask builder from a two-term to a three-term constructor.

3.1 Per-token projections

For each $t \in \{1, \dots, T\}$ compute

$$\mathbf{B}_t = \mathbf{W}_B \mathbf{x}_t \quad (\text{key}) \quad (36)$$

$$\mathbf{C}_t = \mathbf{W}_C \mathbf{x}_t \quad (\text{query}) \quad (37)$$

$$\Delta_t = \text{Softplus}(\mathbf{w}_\Delta^\top \mathbf{x}_t) \quad (\text{step size}) \quad (38)$$

$$A_t = -\text{Softplus}(\mathbf{w}_A^\top \mathbf{x}_t) \text{ or const.} \quad (\text{decay}) \quad (39)$$

$$\lambda_t = \sigma(\mathbf{w}_\lambda^\top \mathbf{x}_t) \in [0, 1] \quad (\text{trapezoidal weight}) \quad (40)$$

$$\mathbf{v}_t = \text{SiLU}(\mathbf{W}_V \mathbf{x}_t) \quad (\text{value}) \quad (41)$$

Form $\alpha_t, \beta_t, \gamma_t$ from (12). Optionally apply **BCNorm** (RMSNorm on $\mathbf{B}_t, \mathbf{C}_t$; [1] §3.4) and learnable channel-wise biases $\mathbf{b}_B, \mathbf{b}_C$ before using them.

3.2 Self-attention output

Assemble $\mathbf{B}, \mathbf{C} \in \mathbb{R}^{T \times N}$ and the value-projected matrix $\mathbf{V} = (\mathbf{v}_1, \dots, \mathbf{v}_T)^\top \in \mathbb{R}^{T \times D}$ with rows $\mathbf{v}_t = \text{SiLU}(\mathbf{W}_V \mathbf{x}_t)$ (6), and the mask $\mathbf{L}$ from (25) or (33). The self-attention output is

$$\mathbf{Y}_{\text{self}} = (\mathbf{L} \odot \mathbf{C}\mathbf{B}^\top)\mathbf{V}, \quad (42)$$

or, entrywise,

$$\mathbf{y}_i^{\text{self}} = \sum_{j=1}^i L_{ij} (\mathbf{C}_i^\top \mathbf{B}_j) \mathbf{v}_j. \quad (43)$$

Because $L_{ij} = 0$ for $j > i$, the attention is *causal*: token i sees tokens $1, \dots, i$. We remove this causality in Section 5 by running a second pass in the reverse direction.

3.3 Complex-valued SSM / RoPE trick: attention with learned RoPE

Mamba-3 promotes $\mathbf{A}$ from a real scalar to a block-diagonal matrix of 2×2 rotations

$$\mathbf{R}_t = \text{Block}(\{R(\Delta_t \boldsymbol{\theta}_t[i])\}_{i=1}^{N/2}), \quad R(\phi) = \begin{bmatrix} \cos \phi & -\sin \phi \\ \sin \phi & \cos \phi \end{bmatrix}, \quad (44)$$

and Proposition 4 of [1] shows that this is equivalent to applying data-dependent rotary embeddings to $\mathbf{B}$ and $\mathbf{C}$. Writing $\mathbf{R}_{\leq t} := \prod_{i=0}^t \mathbf{R}_i^\top$, the complex recurrence is

$$\mathbf{h}_t = \alpha_t \mathbf{h}_{t-1} + \beta_t \mathbf{R}_{\leq t-1} \mathbf{B}_{t-1} \mathbf{x}_{t-1} + \gamma_t \mathbf{R}_{\leq t} \mathbf{B}_t \mathbf{x}_t, \quad \mathbf{y}_t = (\mathbf{R}_{\leq t} \mathbf{C}_t)^\top \mathbf{h}_t. \quad (45)$$

Following the same unrolling as in Section 2, the analogue of (43) becomes

$$\mathbf{y}_i^{\text{self}} = \sum_{j=1}^i L_{ij} (\tilde{\mathbf{C}}_i^\top \tilde{\mathbf{B}}_j) \mathbf{v}_j, \quad \tilde{\mathbf{C}}_i := \mathbf{R}_{\leq i} \mathbf{C}_i, \quad \tilde{\mathbf{B}}_j := \mathbf{R}_{\leq j} \mathbf{B}_j. \quad (46)$$

That is, **Mamba-3 self-attention is linear attention with a structured mask $\mathbf{L}$ and a *data-dependent* rotary embedding** applied to the $\mathbf{B}$ (key) and $\mathbf{C}$ (query) projections, exactly the Transformer’s RoPE but with angles learned per token rather than fixed to $10000^{-2i/N}$.

3.4 MIMO extension

Mamba-3’s MIMO variant ([1] §3.3, eqs. (12)–(14)) stacks R parallel SSMs that share the scalar gates α_t, Δ_t but carry independent $\mathbf{B}_t^{(j)}, \mathbf{C}_t^{(i)}, \mathbf{x}_t^{(j)}$. The head-level output is

$$\mathbf{y}_t^{(i)} = \sum_{j=0}^{R-1} \text{SSM}(\alpha, \Delta, \mathbf{B}^{(j)}, \mathbf{C}^{(i)}, \mathbf{x}^{(j)})_t, \quad (47)$$

i.e. R^2 SISO calls in parallel. Using (28) on each, the MIMO self-attention output is

$$\mathbf{Y}^{(i)} = \sum_{j=0}^{R-1} (\mathbf{L} \odot \mathbf{C}^{(i)} \mathbf{B}^{(j)\top}) \mathbf{X}^{(j)}, \quad i = 0, \dots, R-1. \quad (48)$$

This is exactly multi-head linear attention with shared mask.

3.5 Worked example: self-attention on one 504×504 image

To make the shapes concrete, apply (42) to a single RGB image $\mathbf{I} \in \mathbb{R}^{504 \times 504 \times 3}$ with patch size $P = 14$ and feature width $D = 384$, split across $H = 6$ heads ($d_{\text{head}} = 64$) with state dimension $N = 64$. No special tokens, no multi-view assembly — just the operator on one image.

From pixels to tokens. The 504×504 RGB image is sliced into non-overlapping 14×14 patches by a single strided convolution (*patch embedding*), giving a 36×36 grid of patch-tokens. Each patch is then mapped to a D -dim feature vector by exactly the linear map of (72); we make every step concrete here.

Step 1 — slice the image into patches. Index the patch grid by (r, c) with $r, c \in \{1, \dots, 36\}$. The (r, c) -th patch is the $14 \times 14 \times 3$ block

$$\mathbf{P}_{r,c} \in \mathbb{R}^{14 \times 14 \times 3}, \quad \mathbf{P}_{r,c}[i, j, k] = \mathbf{I}[14(r-1)+i, 14(c-1)+j, k]. \quad (49)$$

Flatten its three colour planes into one vector (row-major over height, width, channel):

$$\mathbf{p}_{r,c} := \text{vec}(\mathbf{P}_{r,c}) \in \mathbb{R}^{14 \cdot 14 \cdot 3} = \mathbb{R}^{588}. \quad (50)$$

Each pixel of the patch contributes 3 scalars (one per RGB channel), so a patch is a 588-dim vector of raw pixel intensities (typically each rescaled to $[0, 1]$ and then ImageNet-normalised).

Step 2 — project each patch to a D -dim feature. A single learned linear map (the patch-embedding weight) is shared across all 1296 patches:

$$\mathbf{x}_{r,c} = \mathbf{W}_{\text{emb}} \mathbf{p}_{r,c} + \mathbf{b}_{\text{emb}} \in \mathbb{R}^D = \mathbb{R}^{384}, \quad \mathbf{W}_{\text{emb}} \in \mathbb{R}^{384 \times 588}, \quad \mathbf{b}_{\text{emb}} \in \mathbb{R}^{384}. \quad (51)$$

This is (72) applied to our concrete numbers. Implemented as a $\text{Conv2d}(3 \rightarrow D, \text{kernel}=14, \text{stride}=14)$ on the image, it does the slice and the linear map in one pass — mathematically identical to the per-patch formula above. The total parameter count of this layer is $D \cdot 14 \cdot 14 \cdot 3 + D = 384 \cdot 588 + 384 = 226\,176$ weights, shared across every patch position; learning adjusts $\mathbf{W}_{\text{emb}}$ so that the D -dim output discriminates whatever the downstream loss requires (class identity, depth, semantic mask, etc.).

Step 3 — flatten the patch grid to a token sequence. Read the 36×36 grid in row-major order to produce a length- T sequence:

$$\mathbf{x}_t := \mathbf{x}_{r,c}, \quad t = 36(r-1) + c, \quad t = 1, \dots, T, \quad T = 36 \cdot 36 = 1296. \quad (52)$$

Stacking the T rows gives the token matrix consumed by the attention layer:

$$\mathbf{X} = (\mathbf{x}_1, \dots, \mathbf{x}_T)^\top \in \mathbb{R}^{T \times D} = \mathbb{R}^{1296 \times 384}. \quad (53)$$

The attention layer never sees pixels — it sees these 1296 token vectors, each of dimension $D = 384$. Every subsequent operation in this worked example (projections to $\mathbf{B}, \mathbf{C}, \mathbf{V}$, the masked inner product, etc.) treats $\mathbf{X}$ as its input matrix, with no further reference to the original 504×504 pixel grid.

What the layer computes. The per-token projections (1), (2), (6), (4) produce

$$\mathbf{B}, \mathbf{C} \in \mathbb{R}^{T \times N} = \mathbb{R}^{1296 \times 64}, \quad \mathbf{V} \in \mathbb{R}^{T \times D} = \mathbb{R}^{1296 \times 384}, \quad \mathbf{L} \in \mathbb{R}^{T \times T} = \mathbb{R}^{1296 \times 1296}, \quad (54)$$

where $\mathbf{V} = (\mathbf{v}_1, \dots, \mathbf{v}_T)^\top$ collects the value-projected rows $\mathbf{v}_t = \text{SiLU}(\mathbf{W}_V \mathbf{x}_t)$. The output is

$$\mathbf{Y}_{\text{self}} = (\mathbf{L} \odot \mathbf{C} \mathbf{B}^\top) \mathbf{V} \in \mathbb{R}^{T \times D} = \mathbb{R}^{1296 \times 384}. \quad (55)$$

Output shape equals input shape: every patch still has a single D -dimensional feature; only its content has been re-mixed across the 1296 tokens, weighted by the structured mask $\mathbf{L}$.

Cost in this configuration. Materializing the masked inner products costs $O(T^2 N) = 1296^2 \cdot 64 \approx 1.07 \times 10^8$ multiply-adds per head; the value mixing then costs $O(T^2 d_{\text{head}}) = 1296^2 \cdot 64 \approx 1.07 \times 10^8$ per head. The $T \times T$ tensor itself is $1296^2 \approx 1.68 \times 10^6$ scalars (~ 6.7 MiB at fp32 per head), which is the term that scales quadratically with image side.

Causality on a 2-D grid. $\mathbf{L}_{ij} = 0$ for $j > i$ in (25)/(33): a patch in the top-right of the grid never reads from the bottom-left. To use this operator as a usable image self-attention, run it on the forward and reverse permutations and sum, as in (74)/(75); that is what Section 5.3 adds.

4 Cross-Attention via Mamba-3

For cross-attention we have two token sequences: a *query* sequence $\mathbf{X}^q \in \mathbb{R}^{T_q \times D}$ and a *key/value* (context) sequence $\mathbf{X}^{kv} \in \mathbb{R}^{T_{kv} \times D}$.

Why cross-attention needs no new derivation. Cross-attention reuses the same identity, just with two different sequences feeding the two sides instead of one. Place the two formulas side by side:

$$\underbrace{Y = (\mathbf{L}^{\text{cross}} \odot \mathbf{C}^q \mathbf{B}^{kv\top}) \mathbf{V}^{kv}}_{\text{Mamba-3 cross-SSD}} \iff \underbrace{Y = \text{softmax}(\mathbf{Q} \mathbf{K}^\top / \sqrt{d}) \mathbf{V}}_{\text{softmax cross-attention}},$$

with the substitution

$$\mathbf{Q} \leftarrow \mathbf{C}^q, \quad \mathbf{K} \leftarrow \mathbf{B}^{kv}, \quad \mathbf{V} \leftarrow \mathbf{V}^{kv} = \text{SiLU}(\mathbf{W}_V \mathbf{x}^{kv}).$$

The cross-attention output equation arises from (28) by the same bridge as (35): the input to the SSD kernel is the kv-side value matrix $\mathbf{V}^{kv} \in \mathbb{R}^{T_{kv} \times D}$, so the placeholder $\mathbf{X}$ in (28) is filled by $\mathbf{V}^{kv}$. The reading rule: the *query* projection comes from the query-side tokens $\mathbf{x}^q$, while *both* the key and the value projections come from the kv-side tokens $\mathbf{x}^{kv}$. Nothing about the operator changes — only which sequence each projection reads from.

Two things that do change relative to self-attention. First, the mask $\mathbf{L}^{\text{cross}}$ (64) is rectangular ($T_q \times T_{kv}$) rather than triangular, because queries and keys live on different time axes and there is no notion of one token coming "before" another across the two streams. Second, the trapezoidal structure of (33) carries over to the kv axis when the kv side is itself produced by a Mamba-3 recurrence (the typical case in Variant B, §4.3): each column of $\mathbf{L}^{\text{cross}}$ then has the same two-band decay profile as a self-SSD row, with the recency bias discussed in §4.3. The output formula $\mathbf{Y} = (\mathbf{L}^{\text{cross}} \odot \mathbf{C}^q \mathbf{B}^{kv\top}) \mathbf{V}^{kv}$ is *unchanged*; only the geometry of $\mathbf{L}$ and which tokens supply $\mathbf{C}, \mathbf{B}, \mathbf{V}$ are different. Consequence for an implementer: cross-attention reuses every line of code that handles self-SSD, with two changes — feed $\mathbf{C}$ from the query sequence and $\mathbf{B}, \mathbf{V}$ from the kv sequence, and swap the triangular mask builder for the rectangular **cross** mask.

4.1 Projections for the two sides

Query side:

$$\mathbf{C}_i^q = \mathbf{W}_C \mathbf{x}_i^q, \quad i = 1, \dots, T_q, \quad (56)$$

that is, only the *query* projection is taken from the query side. Key/value side:

$$\mathbf{B}_j^{kv} = \mathbf{W}_B \mathbf{x}_j^{kv}, \quad (57)$$

$$\Delta_j^{kv} = \text{Softplus}(\mathbf{w}_\Delta^\top \mathbf{x}_j^{kv}), \quad (58)$$

$$A_j^{kv} = -\text{Softplus}(\mathbf{w}_A^\top \mathbf{x}_j^{kv}) \text{ or const.}, \quad (59)$$

$$\mathbf{v}_j^{kv} = \text{SiLU}(\mathbf{W}_V \mathbf{x}_j^{kv}), \quad j = 1, \dots, T_{kv}. \quad (60)$$

Decay scalars α_j^{kv} etc. are computed as in (12) but using the *kv*-side Δ_j^{kv}, A_j^{kv} .

4.2 Variant A: state-compressed cross-attention (constant memory)

Run the Mamba-3 recurrence (13) on the *kv*-side with *outer-product* accumulation, treating $\mathbf{v}_j^{kv}$ as the input instead of $\mathbf{x}_j^{kv}$. Define a matrix-valued hidden state $\mathbf{H}_j \in \mathbb{R}^{N \times D}$ by

$$\mathbf{H}_j = \alpha_j^{kv} \mathbf{H}_{j-1} + \gamma_j^{kv} \mathbf{B}_j^{kv} \mathbf{v}_j^{kv\top}, \quad \mathbf{H}_0 = \mathbf{0}. \quad (61)$$

By the exact same unrolling as in (17) we obtain

$$\mathbf{H}_{T_{kv}} = \sum_{j=1}^{T_{kv}} \left(\prod_{k=j+1}^{T_{kv}} \alpha_k^{kv} \right) \gamma_j^{kv} \mathbf{B}_j^{kv} \mathbf{v}_j^{kv\top} =: \mathbf{h}_{\text{ref}} \in \mathbb{R}^{N \times D}. \quad (62)$$

This $\mathbf{h}_{\text{ref}}$ is the *compressed context*: it condenses the entire kv -sequence into a single $N \times D$ matrix. For each query $\mathbf{C}_i^q$, the output is obtained by a single left-multiplication:

$$\boxed{\mathbf{y}_i^{\text{cross-A}} = \mathbf{C}_i^{q\top} \mathbf{h}_{\text{ref}} \in \mathbb{R}^D.} \quad (63)$$

Shape check: $\mathbf{C}_i^{q\top} \in \mathbb{R}^{1 \times N}$ times $\mathbf{h}_{\text{ref}} \in \mathbb{R}^{N \times D}$ gives $\mathbb{R}^{1 \times D}$, i.e. a D -vector.

Cost. Once $\mathbf{h}_{\text{ref}}$ is built in $O(T_{kv}ND)$ time, answering all T_q queries costs $O(T_qND)$. Memory for the “KV cache” is constant in T_{kv} (it is exactly $\mathbf{h}_{\text{ref}} \in \mathbb{R}^{N \times D}$). This is the analogue of the recurrent KV cache.

4.3 Variant B: token-level cross-attention (exact SSD)

To allow every query token to see every kv token independently, keep the full similarity matrix. Build an $T_q \times T_{kv}$ mask

$$L_{ij}^{\text{cross}} = \gamma_j^{kv} \left(\prod_{k=j+1}^{T_{kv}} \alpha_k^{kv} \right), \quad (64)$$

whose j -th column is the same decay profile as in (25), but now the matrix is *not triangular* because queries do not live on the same time axis as keys. The cross-attention output is

$$\boxed{\mathbf{Y}^{\text{cross-B}} = (\mathbf{L}^{\text{cross}} \odot \mathbf{C}^q \mathbf{B}^{kv\top}) \mathbf{V}^{kv},} \quad (65)$$

or, entrywise,

$$\mathbf{y}_i^{\text{cross-B}} = \sum_{j=1}^{T_{kv}} L_{ij}^{\text{cross}} (\mathbf{C}_i^{q\top} \mathbf{B}_j^{kv}) \mathbf{v}_j^{kv}. \quad (66)$$

Compare with softmax cross-attention $\mathbf{Y} = \text{softmax}(\mathbf{C}^q \mathbf{B}^{kv\top}) \mathbf{V}^{kv}$: (65) replaces softmax row normalization with an elementwise, structured, learned mask. With the trivial mask $L_{ij}^{\text{cross}} \equiv 1$ we obtain ordinary linear cross-attention $\mathbf{C}^q \mathbf{B}^{kv\top} \mathbf{V}^{kv}$.

End-of-sequence bias in $\mathbf{L}^{\text{cross}}$. Note that (64) does not depend on i : every query row is the same column-decay profile. Concretely, $L_{i,T_{kv}}^{\text{cross}} = \gamma_{T_{kv}}$ while $L_{i,1}^{\text{cross}} = \gamma_1 \prod_{k=2}^{T_{kv}} \alpha_k$ is exponentially small once T_{kv} is large. This encodes a strong *recency bias toward the end of the kv sequence*: every query already prefers the last kv tokens before any data-dependent similarity $\mathbf{C}_i^{q\top} \mathbf{B}_j^{kv}$ is applied. This is the right inductive bias when the kv sequence has a natural causal order (decoder-style cross-attention), but it is arbitrary when the kv sequence is a permutation-invariant set — most notably, when it comes from an image patch grid. The bias becomes the order chosen by the raster scan.

Bidirectional cross-attention for permutation-invariant kv . When the kv stream has no preferred order, replace $\mathbf{L}^{\text{cross}}$ by the symmetrised mask

$$L_{ij}^{\text{cross},\leftrightarrow} = \gamma_j \left[\prod_{k=j+1}^{T_{kv}} \alpha_k + \prod_{k=1}^{j-1} \alpha_k \right], \quad (67)$$

which is obtained by summing (64) with its reverse-order counterpart, analogously to (74). The two products coincide at the centre of the kv sequence and decay symmetrically toward both endpoints, eliminating the raster-order preference while preserving the linear-complexity structure. For images this is the natural cross-attention mask to pair with the bidirectional self-attention of Section 5.3.

4.4 Which variant to use

	Variant A (state-compressed)	Variant B (token-level)
State size	$\mathcal{O}(ND)$	$\mathcal{O}(T_q T_{kv})$ (or streamed)
Compute	$\mathcal{O}((T_{kv} + T_q)ND)$	$\mathcal{O}(T_q T_{kv}(N + D))$
Query adaptivity	Only via $\mathbf{C}_i^q$	Via full $\mathbf{C}_i^{q\top} \mathbf{B}_j^{kv}$
Typical use	Long context / large T_{kv}	Short cross-attention / ablations

Both variants can be augmented with the RoPE trick of §3.3: apply the rotation $\mathbf{R}_{\leq i}^{(q)}$ to each query $\mathbf{C}_i^q$ and $\mathbf{R}_{\leq j}^{(kv)}$ to each key $\mathbf{B}_j^{kv}$ before the inner product. This injects relative positional information into cross-attention.

4.5 Worked example: cross-attention between two 504×504 images

Two RGB images $\mathbf{I}^q, \mathbf{I}^{kv} \in \mathbb{R}^{504 \times 504 \times 3}$ go through the same patch embedding (72) independently, giving two token sequences

$$\mathbf{X}^q \in \mathbb{R}^{T_q \times D} = \mathbb{R}^{1296 \times 384}, \quad \mathbf{X}^{kv} \in \mathbb{R}^{T_{kv} \times D} = \mathbb{R}^{1296 \times 384}. \quad (68)$$

The per-token projections (56) produce a query $\mathbf{C}^q \in \mathbb{R}^{T_q \times N}$ for the first image and a key / value pair $\mathbf{B}^{kv} \in \mathbb{R}^{T_{kv} \times N}$, $\mathbf{X}^{kv} \in \mathbb{R}^{T_{kv} \times D}$ for the second (with $N = 64$, $D = 384$, $H = 6$, $d_{\text{head}} = 64$ as before).

Variant A: state-compressed. Equation (63) collapses the second image into a single rank- N summary and reads it out per query token of the first:

$$\mathbf{H} = (\mathbf{B}^{kv})^\top \mathbf{X}^{kv} \in \mathbb{R}^{N \times D} = \mathbb{R}^{64 \times 384}, \quad (69)$$

$$\mathbf{Y}^q = \mathbf{C}^q \mathbf{H} \in \mathbb{R}^{T_q \times D} = \mathbb{R}^{1296 \times 384}. \quad (70)$$

The summary $\mathbf{H}$ has $N \cdot D = 24\,576$ scalars *independent of how big the second image is*. Doubling $\mathbf{I}^{kv}$'s resolution from 504^2 to 1008^2 ($T_{kv} = 5184$) leaves $\mathbf{H}$'s shape untouched — only the one-time accumulation cost grows linearly. The output for the first image keeps its $\mathbf{X}^q$ -shape: 1296 patches, 384 features each.

Variant B: token-level. Equation (65) keeps the full $T_q \times T_{kv}$ inner-product matrix:

$$\mathbf{Y}_i^q = \sum_{j=1}^{T_{kv}} M_{ij} (\mathbf{C}_i^{q\top} \mathbf{B}_j^{kv}) \mathbf{X}_j^{kv} \Rightarrow \mathbf{Y}^q \in \mathbb{R}^{1296 \times 384}, \quad (71)$$

with intermediate weight tensor of size $1296 \times 1296 \approx 1.68 \times 10^6$ scalars (the same quadratic memory term that appears in self-attention). Variant A trades this off for a one-shot rank- N bottleneck; Variant B keeps per-pair adaptivity.

What the output represents. In both variants the output $\mathbf{Y}^q$ is shape-aligned with $\mathbf{X}^q$: *one D -dimensional vector per patch of the first image*, now informed by the content of the second image. Each output patch is the same image-1 grid location it started in — only its features have been updated by information drawn from image 2. Image 2's grid never appears in $\mathbf{Y}^q$; if you also want image 2 informed by image 1, run the operator a second time with the roles of (q, kv) swapped.

5 Applying Mamba-3 to Images

We now assemble the self- and cross-attention blocks into an image model.

5.1 Patchification

Given an image $\mathbf{I} \in \mathbb{R}^{H \times W \times 3}$, choose a patch size P and define $L = (H/P)(W/P)$ patches $\mathbf{p}_{r,c} \in \mathbb{R}^{3P^2}$ for $r \in \{1, \dots, H/P\}$, $c \in \{1, \dots, W/P\}$. Project with a shared linear map:

$$\mathbf{x}_{r,c} = \mathbf{W}_{\text{emb}} \mathbf{p}_{r,c} + \mathbf{b}_{\text{emb}}, \quad \mathbf{W}_{\text{emb}} \in \mathbb{R}^{D \times 3P^2}. \quad (72)$$

Flattening in row-major order gives a token sequence $\mathbf{x}_1, \dots, \mathbf{x}_L$. The same block works for N_v views stacked as $N_v L$ tokens.

5.2 Why a plain causal scan fails on images

Applying (43) directly to the flattened sequence gives

$$\mathbf{y}_i^{\text{self}} = \sum_{j=1}^i L_{ij} (\mathbf{C}_i^\top \mathbf{B}_j) \mathbf{v}_j, \quad (73)$$

i.e. each patch only sees patches *earlier in raster order*. A patch at the top-right thus never reads from the bottom-left — half the image is missing. An image has no natural causal direction, so this asymmetry is harmful.

5.3 Bidirectional and multi-direction scans

Fix a permutation π of $\{1, \dots, L\}$. Let $\text{SSD}_\pi(\mathbf{X})$ mean: permute rows by π , apply (28), permute back. Because SSD’s only source of order is the mask $\mathbf{L}$, every permutation gives a different, still-valid SSM scan.

Two-direction (forward + reverse). With the identity permutation $\pi_{\rightarrow}$ and its reverse $\pi_{\leftarrow}$,

$$\mathbf{Y}^{\text{bi}} = \text{SSD}_{\pi_{\rightarrow}}(\mathbf{X}) + \text{SSD}_{\pi_{\leftarrow}}(\mathbf{X}). \quad (74)$$

Unrolling: for any pair (i, j) , either $j \leq i$ (covered by the forward pass) or $j > i$ (covered by the reverse pass), so every pair of tokens contributes exactly once, with a mask value depending on $|i - j|$ rather than on the sign of $i - j$. This turns causal SSD into bidirectional linear attention on the raster order, matching the bidirectional Mamba variants used in the literature.

Four-direction (raster + reverse + column-major + reverse). For 2-D structure, also scan along columns. Let $\pi_{\downarrow}$ scan column-by-column top-to-bottom and $\pi_{\uparrow}$ its reverse. Then

$$\mathbf{Y}^{\text{4-dir}} = \text{SSD}_{\pi_{\rightarrow}}(\mathbf{X}) + \text{SSD}_{\pi_{\leftarrow}}(\mathbf{X}) + \text{SSD}_{\pi_{\downarrow}}(\mathbf{X}) + \text{SSD}_{\pi_{\uparrow}}(\mathbf{X}). \quad (75)$$

Each summand uses the same learned weights; only the token order changes, so (75) adds no parameters. This mirrors the four scans of Vision-Mamba / V-Mamba realized under the Mamba-3 SSD kernel.

Equivalence to a symmetric mask. Writing (74) entrywise,

$$\mathbf{y}_i^{\text{bi}} = \sum_{j=1}^L \tilde{L}_{ij} (\mathbf{C}_i^\top \mathbf{B}_j) \mathbf{v}_j, \quad \tilde{L}_{ij} = L_{ij}^{\rightarrow} + L_{ij}^{\leftarrow}, \quad (76)$$

where $L^{\rightarrow}$ is (25) and $L^{\leftarrow}$ is the same matrix built on the reversed sequence. $\tilde{L}$ is *symmetric* in i, j : the forward pass covers $j \leq i$, the reverse pass covers $j \geq i$, and with the diagonal counted twice the bidirectional self-attention has no preferred direction.

5.4 2-D positional encoding via the complex SSM

To encode row and column positions, use the RoPE trick of §3.3 with a *factored* angle schedule. For the n -th channel of the state, split into independent row and column angles

$$\theta_{r,c}[n] = \theta^{\text{row}}[n] \cdot r + \theta^{\text{col}}[n] \cdot c, \quad n = 1, \dots, N/2, \quad (77)$$

where $\theta^{\text{row}}, \theta^{\text{col}}$ are learned or fixed frequency vectors. Replace $\boldsymbol{\theta}_t$ in (44) by $\theta_{r,c}$ for the patch at (r, c) . The resulting $\tilde{\mathbf{C}}_i = \mathbf{R}_{\leq i} \mathbf{C}_i$ and $\tilde{\mathbf{B}}_j$ now carry a 2-D rotary position encoding inside the SSD inner product (46).

5.5 Image self-attention layer

Putting the pieces together, one Mamba-3 self-attention layer for images is:

Algorithm (Mamba-3 image self-attention layer).

1. Input tokens $\mathbf{X} \in \mathbb{R}^{L \times D}$ for one image (already patch-embedded).
2. Compute per-token $\mathbf{B}_t, \mathbf{C}_t, \Delta_t, A_t, \lambda_t, \mathbf{v}_t$ via (1)–(6).
3. Apply BCNorm and add learnable biases $\mathbf{b}_B, \mathbf{b}_C$ ([1] §3.4).
4. Apply 2-D RoPE: $\mathbf{B}_t \leftarrow \mathbf{R}_{\leq t} \mathbf{B}_t, \mathbf{C}_t \leftarrow \mathbf{R}_{\leq t} \mathbf{C}_t$ with $\mathbf{R}$ from (77).
5. Build masks $\mathbf{L}^{\rightarrow}, \mathbf{L}^{\leftarrow}$ from (25) (or (33) for the trapezoidal variant) using $\alpha_t, \beta_t, \gamma_t$.
6. Compute $\mathbf{Y}^{\rightarrow} = (\mathbf{L}^{\rightarrow} \odot \mathbf{C} \mathbf{B}^{\top}) \mathbf{V}$ and $\mathbf{Y}^{\leftarrow}$ on the reversed order; optionally also $\mathbf{Y}^{\downarrow}, \mathbf{Y}^{\uparrow}$.
7. Sum across scans to get $\mathbf{Y}^{\text{self}} \in \mathbb{R}^{L \times D}$.
8. Apply the usual post-processing (output projection + SwiGLU MLP block + residual), exactly as in a Mamba-3 / Llama block ([1] Fig. 2).

5.6 Multi-view cross-attention for 3-D applications

For multi-view inputs (e.g. [2], Fig. 2), stack the tokens of N_v views into $\mathbf{X} \in \mathbb{R}^{N_v L \times D}$. Let $V(i) \in \{1, \dots, N_v\}$ denote the view index of token i .

Within-view layer. Apply the self-attention layer of the previous subsection *per view*: i.e. the mask is $L_{ij}^{\text{in}} = L_{ij} \cdot \mathbb{I}[V(i) = V(j)]$, so tokens only interact inside their own view. This is block-diagonal bidirectional SSD.

Cross-view layer. Treat one view at a time as the *query* side and all other views as the *key/value* side. Build

$$L_{ij}^{\text{cv}} = L_{ij}^{\text{cross}} \cdot \mathbb{I}[V(i) \neq V(j)], \quad (78)$$

with L^{cross} from (64) (where the “time” axis is a learned ordering of views, e.g. a canonical “reference view first” order). The output is

$$\mathbf{Y}^{\text{cv}} = (\mathbf{L}^{\text{cv}} \odot \mathbf{C} \mathbf{B}^{\top}) \mathbf{V}. \quad (79)$$

Alternating within-view and cross-view layers realizes the same design as [2] Fig. 2 (“within-view self-attention” and “cross-view self-attention”), but with softmax attention replaced by the Mamba-3 SSD kernel, inheriting its linear complexity in the token count.

Two-view cross-attention (asymmetric). For the special case of a query image and a reference image, use Variant A of Section 4.2: build $\mathbf{h}_{\text{ref}} \in \mathbb{R}^{N \times D}$ from the reference tokens once, and answer each query token in $O(ND)$. This is the natural “Mamba-3 cross-attention” block when the reference view is much larger than the query view, or is reused across many queries (e.g. a novel-view-synthesis decoder reading from a shared scene encoding).

6 Non-Causal SSD and the VSSD Image Model

The bidirectional and multi-direction scans of §5.3 remove causality by *repeating* the SSD kernel along several permutations of the token order. [3] (VSSD, ICCV 2025) proposes an alternative that removes causality *at the algebraic level*, so a single forward pass already yields a non-causal, position-independent operator. We derive their construction from the two-term recurrence (13) using only the algebra of Section 2, then describe the resulting image block.

6.1 The reinterpretation of A_t

Recall (13), $\mathbf{h}_t = \alpha_t \mathbf{h}_{t-1} + \gamma_t \mathbf{B}_t \mathbf{x}_t$ with $\alpha_t = e^{\Delta_t A_t} \in (0, 1)$ controlling *how much of the previous state is retained*. [3] observe that in the SSD form, A_t is a learned, data-dependent scalar with no a priori meaning: one is equally free to make it modulate the *current* contribution instead. Concretely, fix the previous-state multiplier to 1 and divide the input weight by A_t :

$$\mathbf{h}_t = \mathbf{h}_{t-1} + \frac{1}{A_t} \mathbf{B}_t \mathbf{x}_t, \quad \mathbf{h}_0 = \mathbf{0}, \quad (80)$$

which is [3] eq. (7). Unrolling (80) with $\mathbf{h}_0 = \mathbf{0}$,

$$\mathbf{h}_t = \sum_{i=1}^t \frac{1}{A_i} \mathbf{B}_i \mathbf{x}_i. \quad (81)$$

Compared with (17), the multiplicative decay $\prod_{k=j+1}^t \alpha_k$ has been replaced by a flat 1: a token’s contribution to the state no longer depends on *when* it arrives, only on its own coefficient $1/A_i$. The mechanism is now *self-referential*: each token decides its own weight.

6.2 Bidirectional NC-SSD collapses to a global hidden state

The recurrence (81) is still causal in the sense that $\mathbf{h}_t$ sums only over $i \leq t$. Run it once forward and once backward and sum, exactly as in (74), defining $\mathbf{Z}_j := \mathbf{B}_j \mathbf{x}_j^\top \in \mathbb{R}^{N \times D}$ (the outer product appearing in (61)):

$$\mathbf{H}_i^{\rightarrow} = \sum_{j=1}^i \frac{1}{A_j} \mathbf{Z}_j, \quad (82)$$

$$\mathbf{H}_i^{\leftarrow} = \sum_{j=i}^L \frac{1}{A_j} \mathbf{Z}_j, \quad (83)$$

$$\mathbf{H}_i^{\text{bi}} = \mathbf{H}_i^{\rightarrow} + \mathbf{H}_i^{\leftarrow} - \frac{1}{A_i} \mathbf{Z}_i = \sum_{j=1}^L \frac{1}{A_j} \mathbf{Z}_j. \quad (84)$$

The $-\frac{1}{A_i} \mathbf{Z}_i$ correction removes the double counting of $j = i$. The last equality is the key observation of [3]: the bidirectional NC-SSD hidden state is *independent of i* . Writing

$$\mathbf{H} := \sum_{j=1}^L \frac{1}{A_j} \mathbf{Z}_j = \sum_{j=1}^L \frac{1}{A_j} \mathbf{B}_j \mathbf{x}_j^\top \in \mathbb{R}^{N \times D}, \quad (85)$$

every output token reads from the *same* compressed context $\mathbf{H}$. [3] eq. (8) further notes that the self-term $\frac{1}{A_i}\mathbf{Z}_i$ is redundant with the block’s skip connection, so in practice one keeps the simpler global form (85) and skips the correction.

6.3 Closed form: $\mathbf{Y} = \mathbf{C}((\mathbf{B} \odot \mathbf{m})^\top \mathbf{X})$

Let $\mathbf{m} \in \mathbb{R}^L$ with $m_j := 1/A_j$, and write $\mathbf{B} \odot \mathbf{m}$ for the matrix whose j -th row is $m_j \mathbf{B}_j$ (i.e. $\mathbf{m}$ broadcast along the N -axis). The output equation $\mathbf{y}_t = \mathbf{C}_t^\top \mathbf{H}$ stacks to

$$\boxed{\mathbf{Y} = \mathbf{C} \mathbf{H} = \mathbf{C}((\mathbf{B} \odot \mathbf{m})^\top \mathbf{X})}, \quad (86)$$

which is [3] eq. (10). Shape check: $(\mathbf{B} \odot \mathbf{m})^\top \in \mathbb{R}^{N \times L}$, $\mathbf{X} \in \mathbb{R}^{L \times D}$, product is $\mathbb{R}^{N \times D}$, and $\mathbf{C} \in \mathbb{R}^{L \times N}$ times that is $\mathbb{R}^{L \times D}$, as required.

Comparing (86) with the SSD identity (28),

$$\mathbf{Y}_{\text{SSD}} = (\mathbf{L} \odot \mathbf{C} \mathbf{B}^\top) \mathbf{X}, \quad \mathbf{Y}_{\text{NC-SSD}} = \mathbf{C}((\mathbf{B} \odot \mathbf{m})^\top \mathbf{X}), \quad (87)$$

the $T \times T$ structured mask $\mathbf{L}$ has been replaced by a T -vector $\mathbf{m}$ that column-scales $\mathbf{B}$. The two operations are no longer of the same shape:

- SSD pays $O(T^2 N)$ to form $\mathbf{L} \odot \mathbf{C} \mathbf{B}^\top$, then $O(T^2 D)$ to multiply by $\mathbf{X}$, with state of size $\mathbf{L}$ ($\Theta(T^2)$ if materialized).
- NC-SSD pays $O(TND)$ to form $\mathbf{H}$, then $O(TND)$ for $\mathbf{C} \mathbf{H}$, with state of size $\mathbf{H} \in \mathbb{R}^{N \times D}$ ($O(ND)$, independent of T).

The price is uniformity: every query reads the same global summary $\mathbf{H}$, so the per-pair relative weight that $\mathbf{L}_{ij}$ encodes in SSD is gone. [3] argues this is a correct trade for images, where no 1-D distance is meaningful anyway.

Practical note on parameterization. [3] §3.2 remarks that since A_t and $1/A_t$ span the same range of values up to inversion, it is numerically simpler to learn $\mathbf{m}$ directly (equivalently, to learn A via the existing projection (4) and then treat its output as $\mathbf{m}$). The two parameterizations differ only in the optimization geometry.

6.4 Relation to Mamba-3

VSSD is built on Mamba-2 (scalar A , two-term recurrence). The non-causality argument carries over to Mamba-3 unchanged: at the SSD level, both Mamba-2’s (25) and Mamba-3’s (33) have the same shape $\mathbf{Y} = (\mathbf{L} \odot \mathbf{C} \mathbf{B}^\top) \mathbf{X}$, and setting $\alpha_t \equiv 1$, $\gamma_t \mapsto 1/A_t$, $\beta_t \equiv 0$ turns (33) into the rank-one mask $\mathbf{L}_{ij} = m_j$, which factors out of the Hadamard product:

$$\mathbf{L}_{ij} = m_j \implies \mathbf{L} \odot \mathbf{C} \mathbf{B}^\top = \mathbf{C} (\mathbf{B} \odot \mathbf{m})^\top, \quad (88)$$

recovering (86). The Mamba-3 extras compose cleanly on top of NC-SSD:

- The RoPE trick of §3.3 still applies: rotate $\mathbf{B}_j, \mathbf{C}_i$ by $\mathbf{R}_{\leq j}, \mathbf{R}_{\leq i}$ before (86), and the inner product $\tilde{\mathbf{C}}_i^\top \tilde{\mathbf{B}}_j$ carries relative positional information. With the 2-D factorization (77), this becomes the 2-D RoPE used by VSSD.
- The MIMO extension of §3.4 carries over by replicating (86) per head: $\mathbf{Y}^{(i)} = \mathbf{C}^{(i)}((\mathbf{B}^{(j)} \odot \mathbf{m})^\top \mathbf{X}^{(j)})$, summed over j .

The trapezoidal three-term form (11) does *not* survive the NC reduction: β_t implements a two-band relative bias that fundamentally depends on i, j separation, which the single-vector $\mathbf{m}$ cannot represent. VSSD-style NC-SSD is thus strictly a Mamba-2-style reduction of Mamba-3 — useful when the mask’s expressivity is not needed (Section 6.8).

6.5 Tensor-contraction algorithm

[3] eq. (9) writes (86) as three einsum contractions, which matches the standard SSD kernel up to one collapsing step:

$$\mathbf{Z} = \text{contract}(LD, LN \rightarrow LND)(\mathbf{X}, \mathbf{B}), \quad (89)$$

$$\mathbf{H} = \text{contract}(L, LND \rightarrow ND)(\mathbf{m}, \mathbf{Z}), \quad (90)$$

$$\mathbf{Y} = \text{contract}(LN, ND \rightarrow LD)(\mathbf{C}, \mathbf{H}). \quad (91)$$

Step (89) expands each token to its outer product $\mathbf{B}_j \mathbf{x}_j^\top$; step (90) reduces the L axis into the single global state $\mathbf{H}$; step (91) reads the state out per query. Compared with SSD’s $\text{contract}(LL, LDN \rightarrow ND)$ ([3] eq. (9), top), the LL mask has degenerated to an L -vector and the algorithm has no explicit $T \times T$ tensor at any point.

6.6 The VSSD image block

The NC-SSD operator (86) replaces the SSD kernel inside an otherwise standard vision block. Following [3] §3.3:

1. **Local Perception Unit (LPU)** [3]: a depthwise 3×3 convolution applied to the patch grid before the NC-SSD and again before the FFN, injecting local 2-D inductive bias on top of the global, position-independent NC-SSD context.
2. **Depth-wise conv 1-D $\rightarrow$ 2-D.** The causal Conv1D used inside Mamba-2 between the input and the SSD kernel is replaced by a non-causal depth-wise Conv2D with kernel size 3, consistent with the model being non-causal.
3. **2-D RoPE on $\mathbf{B}, \mathbf{C}$.** The factored angle (77) is applied to $\mathbf{C}_i$ and $\mathbf{B}_j$ before the inner product in (86), restoring the spatial relationship between patches that is otherwise lost in the permutation-invariant global state $\mathbf{H}$. This is the Mamba-3 RoPE trick of §3.3 applied to the NC-SSD kernel.
4. **FFN, residual, normalization.** A standard Transformer-style FFN follows the NC-SSD; both the NC-SSD and the FFN are wrapped in residual connections, matching the ViT / Swin block layout rather than Mamba’s pre-norm SwiGLU layout.
5. **Hybrid with MSA in the last stage.** The four-stage backbone uses NC-SSD blocks in stages 1–3 and standard softmax MSA in the last stage. The rationale is that early stages have many tokens, where the $O(T)$ scaling of NC-SSD dominates; the last stage has few tokens but high-level semantics, where the per-pair adaptivity of softmax MSA pays off.
6. **Overlapped downsampling** between stages, in the style of [3], replaces the non-overlapped patch merging of VMamba.

The vector $\mathbf{m}$ as a saliency map. Because $\mathbf{m} \in \mathbb{R}^L$ acts as the only token-dependent weight in (86), it can be visualized as a 2-D heat map over the patch grid. Empirically [3] report that $\mathbf{m}$ concentrates on the foreground object of the image — i.e. NC-SSD ends up learning a saliency-like weighting that the SSD mask $\mathbf{L}$ never makes explicit.

6.7 Worked example: VSSD on one 504×504 image

Apply (86) to the same single image $\mathbf{I} \in \mathbb{R}^{504 \times 504 \times 3}$, patch $P = 14$, $D = 384$, $H = 6$ heads ($d_{\text{head}} = 64$), state $N = 64$, so that the input to the attention layer is the patch-token matrix $\mathbf{X} \in \mathbb{R}^{T \times D} = \mathbb{R}^{1296 \times 384}$.

Per-token quantities. The projections produce

$$\mathbf{B}, \mathbf{C} \in \mathbb{R}^{T \times N} = \mathbb{R}^{1296 \times 64}, \quad \mathbf{V} \in \mathbb{R}^{T \times D} = \mathbb{R}^{1296 \times 384}, \quad \mathbf{m} = \text{softplus}(\mathbf{A}_{\log}) \in \mathbb{R}_{>0}^T = \mathbb{R}_{>0}^{1296}. \quad (92)$$

$\mathbf{V}$ stacks the value-projected rows $\mathbf{v}_t = \text{SiLU}(\mathbf{W}_V \mathbf{x}_t)$ — the matrix that enters the NC-SSD identity (86) (filling its placeholder $\mathbf{X}$, in the sense of the bridge of §3). The crucial step is that $\mathbf{m}$ is a *single positive scalar per patch* — independent of the query position. This is the degeneration of the $T \times T$ mask $\mathbf{L}$ into a single T -vector that [3] exploits.

Hidden state — a global summary of the image. The contracted form (90) sums all 1296 patches into one small object:

$$\mathbf{H} = \sum_{t'=1}^T (\mathbf{B}_{t'} \mathbf{m}_{t'}) \mathbf{v}_{t'}^\top \in \mathbb{R}^{N \times D} = \mathbb{R}^{64 \times 384}. \quad (93)$$

Total size: $N \cdot D = 24\,576$ scalars regardless of T — the same 24 576 numbers whether the image is 224^2 ($T=256$), 504^2 ($T=1296$), or 1008^2 ($T=5184$). $\mathbf{H}$ is a rank- N compressed picture of the image, weighted by the saliency $\mathbf{m}$.

Per-patch readout. Each output patch then reads from the global summary:

$$\mathbf{Y}_t = \mathbf{C}_t \mathbf{H} \in \mathbb{R}^D, \quad \mathbf{Y} \in \mathbb{R}^{T \times D} = \mathbb{R}^{1296 \times 384}. \quad (94)$$

Output shape equals input shape: every patch still has one D -dim feature. The only patch-specific computation at read-out is the small product $\mathbf{C}_t \mathbf{H}$ ($N \cdot D$ multiply-adds per token). The expensive part — building $\mathbf{H}$ — was paid once.

Comparing the three operators on this image. On the same $T=1296$ tokens and same N, D :

	SSD (Mamba-3)	Softmax MSA	NC-SSD (VSSD)
Per-pair coupling	$\mathbf{L}_{ij} \cdot \mathbf{C}_i^\top \mathbf{B}_j$	$\text{softmax}(\mathbf{Q}_i^\top \mathbf{K}_j)$	m_j only
State size	$\Theta(T^2)$ if $\mathbf{L}$ materialized	$\Theta(T^2)$	$\Theta(ND) = 24\,576$
Cost (this image)	$O(T^2 N) \approx 1.07 \times 10^8$	$O(T^2 D) \approx 6.4 \times 10^8$	$O(TND) \approx 3.2 \times 10^7$
Scaling with image side	quadratic in T	quadratic in T	linear in T

The NC-SSD column is roughly $20\times$ cheaper than softmax MSA on this image and stays linear when the image grows; the cost paid is that every output position reads from the *same* summary $\mathbf{H}$, with the only token-dependent weight being the scalar $m_{t'}$ in (93).

6.8 When to use NC-SSD versus bidirectional SSD

	Bidirectional SSD (§5.3)	NC-SSD (this section)
Mask state	$O(T^2)$ (or banded fused kernel)	$O(ND)$
Compute per layer	$O(T^2(N + D)) \cdot k_{\text{scans}}$	$O(TND)$
Per-pair adaptivity	$\mathbf{L}_{ij}$ and $\mathbf{C}_i^\top \mathbf{B}_j$	$\mathbf{C}_i^\top \mathbf{B}_j$ only
Position encoding	decay in $ i - j $ + RoPE	RoPE only (decay collapses)
Typical regime	moderate T , structure-rich tokens	long T , vision-style tokens

The two are not mutually exclusive: a hybrid stack can use NC-SSD in the early, high-resolution stages where T dominates the cost, and bidirectional SSD (or softmax MSA, as in VSSD) in deeper stages where the mask’s per-pair weighting matters. For our multi-view setting (§5.6), NC-SSD also offers a free state-compressed cross-view block: each view’s NC-SSD hidden state $\mathbf{H}^{(v)} \in \mathbb{R}^{N \times D}$ is already in the form that (63) consumes, so cross-view attention reduces to $\mathbf{C}^q{}^\top \sum_v \mathbf{H}^{(v)}$ with no additional kernel.

7 Application: From the Token Tensor to a Segmentation Mask

The attention layers of Sections 3–5 and the NC-SSD variant of Section 6.4 all output a token tensor of the same shape as their input, $\mathbf{Y} \in \mathbb{R}^{T \times D}$ (for H, W, P, D, T as in §3.5, the working example is $T = 1296$, $D = 384$). To produce a pixel-resolution semantic segmentation mask we attach a separate *segmentation head* on top of $\mathbf{Y}$. This section spells out three standard heads with explicit shape annotations; the choice of head is independent of which attention variant produced $\mathbf{Y}$.

Let K denote the number of semantic classes and assume the attention backbone consumes images of size $H \times W$ with patch size P so the grid is $H_p \times W_p = (H/P) \times (W/P)$ and $T = H_p W_p$.

7.1 Head A: linear projection + bilinear upsample

Following the Segmenter family [3]-style minimal head:

$$\mathbf{Y} \in \mathbb{R}^{T \times D} \xrightarrow{\text{reshape}} \mathbf{Y}^{\text{grid}} \in \mathbb{R}^{D \times H_p \times W_p}, \quad (95)$$

$$\mathbf{Z} = \text{Conv2D}_{1 \times 1}^{D \rightarrow K}(\mathbf{Y}^{\text{grid}}) \in \mathbb{R}^{K \times H_p \times W_p}, \quad (96)$$

$$\hat{\mathbf{S}} = \text{BilinearUpsample}_{\times P}(\mathbf{Z}) \in \mathbb{R}^{K \times H \times W}, \quad (97)$$

$$\hat{m}(h, w) = \arg \max_k \hat{\mathbf{S}}_{k, h, w}. \quad (98)$$

For the running example $T = 1296$, $D = 384$, $P = 14$, $K = 21$ (PASCAL VOC):

$$\mathbf{Y}_{\text{grid}} : (1, 384, 36, 36) \rightarrow \mathbf{Z} : (1, 21, 36, 36) \rightarrow \hat{\mathbf{S}} : (1, 21, 504, 504). \quad (99)$$

Trainable parameters: just the 1×1 convolution $\mathbf{W} \in \mathbb{R}^{K \times D}$ ($DK = 384 \cdot 21 = 8064$ weights). The head is essentially a per-patch linear classifier followed by deterministic upsampling.

Loss. Train with per-pixel cross-entropy against the ground-truth label map $m^* \in \{1, \dots, K\}^{H \times W}$, optionally augmented with Dice loss for class balance:

$$\mathcal{L}_{\text{seg}} = -\frac{1}{HW} \sum_{h, w} \log \frac{\exp \hat{\mathbf{S}}_{m^*(h, w), h, w}}{\sum_k \exp \hat{\mathbf{S}}_{k, h, w}} + \lambda \mathcal{L}_{\text{Dice}}(\hat{\mathbf{S}}, m^*). \quad (100)$$

7.2 Head B: multi-scale DPT-style decoder

When the attention backbone is deep (e.g. 12 blocks), tap intermediate token tensors from layers $\ell_1 < \ell_2 < \ell_3 < \ell_4$ — call them $\mathbf{Y}^{(\ell_k)} \in \mathbb{R}^{T \times D}$. Reassemble each into a $D \times H_p \times W_p$ grid and feed into a top-down fusion pyramid (*Reassemble* + *Fusion* blocks, identical in structure to DA3’s Dual-DPT but with a per-pixel classifier head instead of a depth regressor). At each pyramid level the spatial resolution doubles via transposed convolution, while D shrinks:

$$\mathbf{F}^{(4)} \in \mathbb{R}^{C \times H_p \times W_p} = \mathbb{R}^{256 \times 36 \times 36}, \quad (101)$$

$$\mathbf{F}^{(3)} \in \mathbb{R}^{C/2 \times 2H_p \times 2W_p} = \mathbb{R}^{128 \times 72 \times 72},$$

$$\mathbf{F}^{(2)} \in \mathbb{R}^{C/4 \times 4H_p \times 4W_p} = \mathbb{R}^{64 \times 144 \times 144}, \quad (102)$$

$$\mathbf{F}^{(1)} \in \mathbb{R}^{C/8 \times H \times W} = \mathbb{R}^{32 \times 504 \times 504},$$

$$\hat{\mathbf{S}} = \text{Conv2D}_{1 \times 1}^{C/8 \rightarrow K}(\mathbf{F}^{(1)}) \in \mathbb{R}^{K \times H \times W}. \quad (103)$$

This is strictly more expensive than Head A (millions of parameters in the fusion path) but recovers fine boundary detail that the patch-coarse Head A cannot. The Mamba-3 / VSSD attention operates at the coarse 36×36 resolution throughout; all the high-frequency structure of the final mask is synthesised by the fusion pyramid from the 36×36 token grid.

7.3 Head C: mask-classification with learnable queries

In the MaskFormer / Mask2Former style, introduce Q learnable query tokens $\mathbf{q}_1, \dots, \mathbf{q}_Q \in \mathbb{R}^D$ (e.g. $Q=100$), treat the patch tokens $\mathbf{Y}$ as keys/values, and run a small *cross-attention stack* on top — exactly the Mamba-3 cross-attention of Section 4. Each query then predicts (i) a class probability and (ii) a soft binary mask that picks out which patches belong to it.

Concretely, write $\mathbf{Q} \in \mathbb{R}^{Q \times D}$ for the queries (after a few cross-attention blocks) and split each into two heads:

$$\mathbf{c}_q^{\text{cls}} = \text{MLP}_{\text{cls}}(\mathbf{q}_q) \in \mathbb{R}^{K+1}, \quad (104)$$

$$\mathbf{e}_q^{\text{mask}} = \text{MLP}_{\text{mask}}(\mathbf{q}_q) \in \mathbb{R}^D. \quad (105)$$

The class-conditional masks come from a per-patch dot product:

$$\mathbf{M}_q = \mathbf{Y} \mathbf{e}_q^{\text{mask}} \in \mathbb{R}^T, \quad \mathbf{M} \in \mathbb{R}^{Q \times T} = \mathbb{R}^{100 \times 1296}. \quad (106)$$

Reshape $\mathbf{M}$ to $\mathbb{R}^{Q \times H_p \times W_p}$, bilinear upsample to $\mathbb{R}^{Q \times H \times W}$, then form the per-class semantic map by marginalising over queries:

$$\hat{\mathbf{S}}_k(h, w) = \sum_{q=1}^Q \text{softmax}_q(\sigma(\mathbf{M}_q(h, w))) \text{softmax}_K(\mathbf{c}_q^{\text{cls}})_k, \quad (107)$$

with σ the sigmoid. Each query effectively claims a region of the patch grid (via $\mathbf{M}_q$) and a class (via $\mathbf{c}_q^{\text{cls}}$); (107) blends those claims at pixel resolution. Trainable parameters are the query embeddings, the two MLPs, and the cross-attention stack — typically $\sim$ few-M parameters, smaller than Head B’s full DPT pyramid but more expressive than Head A.

Loss. Hungarian matching between predicted queries and GT instances, then cross-entropy on $\mathbf{c}^{\text{cls}}$ + per-pixel binary cross-entropy on $\mathbf{M}_q$ (or Dice) for the matched pairs.

7.4 Why the attention variant is interchangeable

Heads A, B, and C all consume $\mathbf{Y} \in \mathbb{R}^{T \times D}$ and nothing more. Whether $\mathbf{Y}$ was produced by (42) (SSD self-attention), (75) (bidirectional SSD), or (86) (NC-SSD / VSSD), the downstream pipeline is identical. The only practical differences are:

- NC-SSD / VSSD’s per-patch saliency $\mathbf{m} \in \mathbb{R}^T$ already concentrates on the foreground (§6.7); this provides a useful *initial* class-agnostic mask that Head C can refine with class identity, which is consistent with Mask2Former’s finding that good initial query-region proposals shorten training.
- Bidirectional SSD’s structured mask $\mathbf{L}_{ij}$ encodes a relative position bias; this is most useful for Head A (which has no other position information at the output) and less critical for Heads B/C (which inject position via the decoder’s spatial structure or query embeddings).
- The cost asymmetry of §6.7 — NC-SSD’s $O(TND)$ vs softmax MSA’s $O(T^2D)$ — propagates: with NC-SSD the bottleneck moves from the backbone to the segmentation head, especially Head B’s pyramid, so high-resolution segmentation benefits disproportionately from VSSD backbones.

8 Application: Multi-Frame 3D Point Tracking

A video clip is a sequence of F RGB frames $\mathbf{I}^{(1)}, \dots, \mathbf{I}^{(F)} \in \mathbb{R}^{H \times W \times 3}$. The point-tracking task is to maintain a set of N persistent *tracks*; for every track $n \in \{1, \dots, N\}$ and every frame $t \in \{1, \dots, F\}$ we predict

- a 3D position $\mathbf{p}_n^{(t)} \in \mathbb{R}^3$ in a shared world frame,
- a visibility / alive flag $\sigma_n^{(t)} \in [0, 1]$ — $\sigma_n^{(t)} \rightarrow 1$ when track n is observed in frame t , $\sigma_n^{(t)} \rightarrow 0$ when it has disappeared (occlusion, out-of-frame, end-of-life), and
- a spawn-confidence $\rho_n^{(t)} \in [0, 1]$ — $\rho_n^{(t)} \rightarrow 1$ on the frame at which a new track is born.

Tracks must keep identity across frames so that $\mathbf{p}_n^{(t)}$ and $\mathbf{p}_n^{(t')}$ are positions of *the same physical 3D point* at times t and t' .

This chapter assembles two Mamba-3 attention blocks of different types: **non-causal VSSD self-attention within each frame** (§6.4), because a single image has no causal direction, and **causal SSD cross-attention across frames** (§4.3), because a track at time t may depend on its history at times $\leq t$ but not on the future. Together they realize a streaming tracker that is linear in patch count and linear in frame count.

8.1 Architecture overview

Block	Operator	Inputs	Output
Per-frame encoder	VSSD self-attn (86)	frame patches $\mathbf{X}^{(t)} \in \mathbb{R}^{T \times D}$	frame features $\mathbf{Y}^{(t)} \in \mathbb{R}^{T \times D}$
Track propagation	causal cross-SSD (65)	queries $\mathbf{Q}^{(t-1)}$ + features $\mathbf{Y}^{(t)}$	updated queries $\mathbf{Q}^{(t)}$
Position head	MLP	$\mathbf{Q}^{(t)}$	$\mathbf{p}^{(t)}, \sigma^{(t)}, \rho^{(t)}$

A *query-conditioned bank* $\mathbf{Q}^{(t)} = (\mathbf{q}_1^{(t)}, \dots, \mathbf{q}_N^{(t)})^\top \in \mathbb{R}^{N \times D}$ carries the identity of every track from one frame to the next. Unlike a free-floating learnable bank, each slot n is *tied to a specific physical point* at construction time: given the user-supplied query (x_n^q, y_n^q, t_n^q) — a pixel location and an anchor frame — the initial bank state is obtained by bilinear-sampling the encoder’s finest pyramid feature at that location:

$$\mathbf{q}_n^{(0)} = \text{BilinearSample}(\mathbf{Y}^{(t_n^q)}, \frac{x_n^q}{W} \cdot W_{\text{grid}}, \frac{y_n^q}{H} \cdot H_{\text{grid}}) \in \mathbb{R}^D, \quad (108)$$

where $W_{\text{grid}} \times H_{\text{grid}}$ is the finest pyramid level’s spatial extent. This eliminates the slot-assignment ambiguity that otherwise arises when free-floating slots are matched to ground-truth tracks via Hungarian assignment: there, the matching flips between batches and the gradient signal becomes incoherent. With (108) the slot identity is fixed by the input, slot n always means *the trajectory of physical point* (x_n^q, y_n^q, t_n^q) , and gradients accumulate coherently per slot.

At each frame t :

1. Encode the frame: $\mathbf{X}^{(t)} \rightarrow \mathbf{Y}^{(t)}$ via the VSSD operator (§8.2).
2. Update the bank: $\mathbf{Q}^{(t-1)} \xrightarrow{\text{cross-SSD over time}} \mathbf{Q}^{(t)}$ using $\mathbf{Y}^{(1)}, \dots, \mathbf{Y}^{(t)}$ as keys / values, with a causal mask that hides future frames (§8.3).
3. Read out 3D position, visibility, and spawn-confidence per track via small MLP heads (§8.4).

8.2 Within-frame block: VSSD self-attention

Each frame t goes through the same VSSD self-attention block as §6.7, independently of the others. Concretely with $H \times W = 504 \times 504$, patch $P = 14$, $D = 384$, $N = 64$:

$$\mathbf{X}^{(t)} \in \mathbb{R}^{T \times D} = \mathbb{R}^{1296 \times 384} \quad (\text{frame-}t \text{ patch tokens}), \quad (109)$$

$$\mathbf{B}^{(t)}, \mathbf{C}^{(t)} \in \mathbb{R}^{T \times N}, \quad \mathbf{V}^{(t)} \in \mathbb{R}^{T \times D}, \quad \mathbf{m}^{(t)} \in \mathbb{R}_{>0}^T, \quad (110)$$

$$\mathbf{H}^{(t)} = (\mathbf{B}^{(t)} \odot \mathbf{m}^{(t)})^\top \mathbf{V}^{(t)} \in \mathbb{R}^{N \times D}, \quad (111)$$

$$\mathbf{Y}^{(t)} = \mathbf{C}^{(t)} \mathbf{H}^{(t)} \in \mathbb{R}^{T \times D} = \mathbb{R}^{1296 \times 384}. \quad (112)$$

The choice of VSSD (non-causal) over bidirectional SSD has two motivations specific to tracking: (i) within a single frame there is no preferred direction in the patch grid, and (ii) the $N \cdot D = 24576$ -scalar global summary $\mathbf{H}^{(t)}$ is independent of T , so high-resolution tracking (large T) does not blow the within-frame budget. Compute per frame is $O(TND)$, memory is $O(ND)$ for the summary.

8.3 Across-frame block: causal cross-SSD

The query bank reads from the frame features via the cross-SSD operator of §4.3, but with two crucial choices:

- (a) **Queries are the persistent bank.** Take $\mathbf{C}^{q,(t)} \in \mathbb{R}^{N \times N_{state}}$ from $\mathbf{Q}^{(t-1)}$, the previous frame’s bank — these are the *tracks asking “where am I now?”* The kv side comes from the new frame’s features $\mathbf{Y}^{(t)} \in \mathbb{R}^{T \times D}$, projected to $\mathbf{B}^{kv,(t)} \in \mathbb{R}^{T \times N_{state}}$ and used as the value matrix in place of $\mathbf{V}^{kv}$. Reading from T patches into N tracks costs $O(NTN_{state})$ in Variant B or $O(TN_{state}D)$ for the state of Variant A.
- (b) **The mask is causal in time, not in patches.** Stack the kv side over the frames seen so far, $\mathbf{Y}^{(1)}, \dots, \mathbf{Y}^{(t)} \in \mathbb{R}^{T \times D}$, and let the cross-mask $\mathbf{L}^{cross} \in \mathbb{R}^{N \times tT}$ block the future:

$$\mathbf{L}_{n,(s,j)}^{cross} = \mathbb{I}[s \leq t] \cdot \left(\prod_{k=s+1}^t \alpha_k \right) \gamma_s, \quad (113)$$

where the second factor is the same temporal decay profile as the SSD mask of (25), but now the running index is over frames rather than tokens, and the cell $\mathbf{L}_{n,(s,j)}^{cross}$ is shared across all patches j of frame s . The indicator $\mathbb{I}[s \leq t]$ enforces temporal causality so a track at time t cannot peek at frames $> t$. This is exactly the recipe of §4.3 adapted from token causality to frame causality.

Concretely, the per-frame update of the bank is

$$\boxed{\mathbf{Q}^{(t)} = (\mathbf{L}^{cross} \odot \mathbf{C}^{q,(t)} \mathbf{B}^{kv,(1:t)})^\top \mathbf{V}^{kv,(1:t)} \in \mathbb{R}^{N \times D}}, \quad (114)$$

where the $(1:t)$ superscript denotes vertical stacking over the first t frames. With the Mamba-3 recurrent form (34) the right-hand side is computed *incrementally*: maintain a frame-level hidden state $\mathbf{h}_{track}^{(t)} \in \mathbb{R}^{N \times N_{state} \times D}$ that is updated in place each frame, so the streaming cost is $O(NN_{state}D)$ per frame — independent of clip length.

8.4 Per-track read-out heads

Each updated query $\mathbf{q}_n^{(t)} \in \mathbb{R}^D$ passes through three small MLPs:

$$\Delta \mathbf{p}_n^{(t)} = \text{MLP}_{xyz}(\mathbf{q}_n^{(t)}) \in \mathbb{R}^3, \quad (\text{motion since query anchor}) \quad (115)$$

$$\sigma_n^{(t)} = \sigma(\text{MLP}_{vis}(\mathbf{q}_n^{(t)})) \in [0, 1], \quad (\text{visibility / alive}) \quad (116)$$

$$\rho_n^{(t)} = \sigma(\text{MLP}_{spawn}(\mathbf{q}_n^{(t)})) \in [0, 1], \quad (\text{spawn flag}) \quad (117)$$

where σ is the sigmoid. The position head predicts $\Delta \mathbf{p}_n^{(t)}$ — the motion of track n at frame t relative to its own query-anchor position — rather than absolute coordinates. A monocular model cannot recover the absolute metric scale of the scene from RGB alone, so any absolute-position prediction is identifiable only up to an unknown per-clip scale factor; predicting the relative trajectory side-steps that ambiguity entirely (the relative motion is well-defined given the camera and the trajectory shape). At inference, the absolute prediction is recovered as $\mathbf{p}_n^{(t)} = \mathbf{p}_n^q + \Delta \mathbf{p}_n^{(t)}$ using the query anchor position $\mathbf{p}_n^q$ (which the TAPVid-3D evaluator provides as the per-track GT position at the query frame).

Stacking over all N tracks gives the predicted trajectory tensor

$$(\Delta \mathbf{p}^{(t)}, \boldsymbol{\sigma}^{(t)}, \boldsymbol{\rho}^{(t)}) \in \mathbb{R}^{N \times 3} \times [0, 1]^N \times [0, 1]^N. \quad (118)$$

8.5 Spawn and disappear

With query-conditioned initialisation (108), N equals the number of user-supplied queries — there is no fixed worst-case slot count to over-allocate, and no need for the Mask2Former-style “no-object” class or Hungarian-spawn matching.

Disappear. A track is treated as occluded at frame t when $\sigma_n^{(t)} < \tau_{\text{vis}}$ (e.g. $\tau_{\text{vis}} = 0.3$). Its query embedding $\mathbf{q}_n^{(t)}$ still updates from the cross-attention, so the track can *re-emerge* if it becomes visible again — the visibility flag is decoupled from the query state.

Spawn / birth frame. The query (x_n^q, y_n^q, t_n^q) specifies when track n is *seeded*; the spawn-confidence head $\rho_n^{(t)}$ predicts when within the clip the underlying physical point first becomes visible, which the loss $\mathcal{L}_{\text{spawn}}$ supervises against $\mathbb{I}[t = t_n^{\text{birth}}]$. New physical points entering the scene after the clip’s first frame are accommodated by adding a new query at the appropriate (x, y, t) ; the bank then grows by one slot without retraining.

8.6 Loss

Because (108) pins slot n to the query (x_n^q, y_n^q, t_n^q) , there is no slot-assignment freedom — slot n is GT track n by construction. No Hungarian matching is needed. Let $\mathbf{p}_n^{*,(t)}, \sigma_n^{*,(t)}$ be the ground-truth 3D position and visibility for that track at frame t , and let $\mathbf{p}_n^q := \mathbf{p}_n^{*,(t_n^q)}$ be the GT 3D position at the query anchor. The ground-truth motion is

$$\Delta \mathbf{p}_n^{*,(t)} := \mathbf{p}_n^{*,(t)} - \mathbf{p}_n^q, \quad (119)$$

and the model’s prediction $\Delta \mathbf{p}_n^{(t)}$ from §8.4 is compared against this.

Per-clip scale normalisation. Different TAPVid-3D subsets span very different metric scales (pstudio / adt are 1–5 m indoor scenes; drivetrack is 10–50 m outdoor driving). An un-normalised L1 on $\Delta \mathbf{p}$ would make a drivetrack batch’s gradient $\sim 10\times$ larger than a pstudio batch’s, swamping the optimisation. Following the TAPVid-3D evaluator’s `scaling=‘median’` convention, normalise both sides by a per-clip scale

$$s = \text{median}(\|\mathbf{p}_n^q\|_2 : n \in \mathcal{N}) + \varepsilon, \quad (120)$$

where $\mathcal{N}$ ranges over the GT tracks in the clip and ε is a small positive constant (e.g. 10^{-3} m). The scaled position residual is then

$$\mathbf{r}_n^{(t)} := \frac{1}{s} (\Delta \mathbf{p}_n^{(t)} - \Delta \mathbf{p}_n^{*,(t)}). \quad (121)$$

Loss terms (v6 — Smooth-L1, magnitude, direction-cosine, 2-D reprojection). A plain ℓ_1 on the scaled residual gives the optimiser the same constant gradient $\pm 1/s$ whether the residual is 1 cm or 1 m. With many TAPVid-3D queries moving only a few cm per 8-frame window, “predict $\Delta \mathbf{p} \approx 0$ ” is a competitive local minimum under ℓ_1 , and the model collapses there. v6 adds three terms that attack the zero-motion baseline and one that grounds the prediction in pixel space:

- **Scale-normalised Smooth- ℓ_1 position** (Huber with $\delta=1$) on visible frames:

$$\mathcal{L}_{\text{pos}} = \frac{1}{Z_{\text{pos}}} \sum_{n,t} \sigma_n^{*,(t)} \sum_{c \in \{x,y,z\}} \text{SmoothL1}(\mathbf{r}_n^{(t)}[c]), \quad (122)$$

with $Z_{\text{pos}} = \sum_{n,t} \sigma_n^{*,(t)}$. Quadratic near zero, linear far away $\Rightarrow$ small residuals receive small gradient, so the optimiser stops wasting capacity reducing already-small errors and focuses on the big-error tracks where “predict 0” is most wrong.

- **Scaled magnitude penalty:**

$$\mathcal{L}_{\text{mag}} = \frac{1}{Z_{\text{pos}}} \sum_{n,t} \sigma_n^{*,(t)} \left(\frac{\|\Delta \mathbf{p}_n^{(t)}\| - \|\Delta \mathbf{p}_n^{*,(t)}\|}{s} \right)^2. \quad (123)$$

Forces the predicted motion to have the right *magnitude ratio to scene scale*, which under a monocular camera is the only magnitude information geometrically recoverable.

- **Direction cosine:**

$$\mathcal{L}_{\text{dir}} = \frac{1}{Z_{\text{pos}}} \sum_{n,t} \sigma_n^{*,(t)} (1 - \cos(\Delta \mathbf{p}_n^{(t)}, \Delta \mathbf{p}_n^{*,(t)})). \quad (124)$$

Penalises wrong-direction motion regardless of magnitude. A model that gets the direction right but under-shoots magnitude is preferred over one that predicts the right magnitude in the wrong direction.

- **2-D pixel-space reprojection.** The TAPVid-3D queries are natively 2-D pixel locations; supervising in pixel space gives the model a geometric signal that is in-distribution with the input image and that has *no* scale ambiguity:

$$\begin{aligned} \hat{\mathbf{p}}_n^{(t)} &= \mathbf{p}_n^q + \Delta \mathbf{p}_n^{(t)}, \quad \hat{\mathbf{u}} \hat{\mathbf{v}}_n^{(t)} = \pi(\hat{\mathbf{p}}_n^{(t)}, \mathbf{K}), \\ \mathcal{L}_{\text{reproj}} &= \frac{1}{Z_{\text{pos}} W_{\text{img}}} \sum_{n,t} \sigma_n^{*,(t)} \sum_{c \in \{u,v\}} \text{SmoothL1}(\hat{\mathbf{u}} \hat{\mathbf{v}}_n^{(t)}[c] - \mathbf{u} \mathbf{v}_n^{*,(t)}[c]), \end{aligned} \quad (125)$$

where $\pi(\cdot, \mathbf{K})$ is pinhole projection through the per-clip intrinsics and W_{img} is the image side length used for training (normalises the pixel residual to $\sim [0, 1]$).

- **Visibility BCE, spawn BCE, and temporal smoothness** unchanged from § v5.

The total loss is

$$\mathcal{L} = \lambda_{\text{pos}} \mathcal{L}_{\text{pos}} + \lambda_{\text{mag}} \mathcal{L}_{\text{mag}} + \lambda_{\text{dir}} \mathcal{L}_{\text{dir}} + \lambda_{\text{reproj}} \mathcal{L}_{\text{reproj}} + \lambda_{\text{vis}} \mathcal{L}_{\text{vis}} + \lambda_{\text{spawn}} \mathcal{L}_{\text{spawn}} + \lambda_{\text{smooth}} \mathcal{L}_{\text{smooth}}, \quad (126)$$

with default weights (1, 0.5, 0.5, 1, 0.5, 0.5, 0.1).

The cross-attention’s causal mask (113) ensures that the loss at frame t depends only on frames $\leq t$; gradients flow back through the recurrent hidden state $\mathbf{h}_{\text{track}}^{(t)}$, so the network learns a streaming update rule, not a teacher-forced batch tracker.

8.7 Why this operator pairing

VSSD self-attention and causal cross-SSD have complementary strengths matched to the two axes of the tracking problem:

- Within a frame, structure is 2-D and orderless — every patch can in principle inform every other. VSSD reaches that all-to-all coverage at $O(T)$ cost via the rank- N global summary $\mathbf{H}^{(t)}$, instead of the $O(T^2)$ that softmax MSA or bidirectional SSD would pay (§6.7’s ratio: $\sim 20\times$ cheaper on a 504^2 image).
- Across frames, structure is 1-D and causally ordered — frame t depends on its history. Causal SSD with the temporal mask (113) preserves that ordering and admits a $O(1)$ -per-frame streaming recurrence (34), so the tracker’s memory does not grow with clip length. The full $\mathbf{L}^{\text{cross}}$ never needs to be materialised at inference.

The persistent query bank $\mathbf{Q}^{(t)}$ is the bridge: it is updated by both blocks (VSSD-encoded features feed it through cross-SSD), but its size $N \times D$ never depends on either the patch count T or the clip length F . The end-to-end forward pass is $O(F \cdot TND)$ in time, $O(NT_{\max}N_{\text{state}}D)$ in memory if the within-frame summaries $\mathbf{H}^{(t)}$ are kept (the default), or $O(NN_{\text{state}}D)$ if only the rolling track hidden state is kept (streaming inference).

9 End-to-End Summary

Using only (28), we have obtained:

- **Self-attention:** (42) and, with learned RoPE, (46). Multi-head via (48).
- **Cross-attention:** Variant A (63) (cheap, compressed KV) and Variant B (65) (exact).
- **Image encoder:** patchify + bidirectional / multi-direction SSD self-attention ((74),(75)) with 2-D RoPE (77).
- **Multi-view encoder:** alternating within-view (79)-style blocks and cross-view (78) blocks; or the compressed-state cross block (63) when one side dominates.
- **Non-causal SSD (VSSD):** reinterpret A_t as a current-token weight to obtain (86), $\mathbf{Y} = \mathbf{C}((\mathbf{B} \odot \mathbf{m})^\top \mathbf{X})$, which collapses the $T \times T$ mask $\mathbf{L}$ to a T -vector $\mathbf{m}$, removes causality without multi-scan, and yields the VSSD image block (Section 6).

Every output formula above is a direct application of the single identity

$$\mathbf{Y} = (\mathbf{L} \odot \mathbf{C}\mathbf{B}^\top)\mathbf{X},$$

which we derived from the Mamba-3 recurrence in Section 2. The choices made by a specific architecture amount to (i) which tokens supply $\mathbf{B}, \mathbf{C}, \mathbf{X}$ (self vs cross vs multi-view), and (ii) which structured mask $\mathbf{L}$ is used (causal vs bidirectional vs block-diagonal vs view-masked). Mamba-3’s exponential-trapezoidal rule (33) and RoPE trick (45) then give the mask and the query/key its final, expressive form.

References

- [1] Aakash Lahoti, Kevin Y. Li, Berlin Chen, Caitlin Wang, Aviv Bick, J. Zico Kolter, Tri Dao, and Albert Gu. Mamba-3: Improved sequence modeling using state space principles. *arXiv preprint arXiv:2603.15569*, 2026.

- [2] Haotong Lin, Sili Chen, Jun Hao Liew, Donny Y. Chen, Zhenyu Li, Guang Shi, Jiashi Feng, and Bingyi Kang. Depth anything 3: Recovering the visual space from any views. *arXiv preprint arXiv:2511.10647*, 2025.
- [3] Yuheng Shi, Mingjia Li, Minjing Dong, and Chang Xu. Vssd: Vision mamba with non-causal state space duality. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2025.
- [4] Biao Zhang and Rico Sennrich. Root mean square layer normalization. *arXiv preprint arXiv:1910.07467*, 2019.